

A Low-Space Algorithm for Worst-Case Subset Sum

OpenAI

September 26, 2026

Abstract

We give a uniform classical randomized decision algorithm for worst-case Subset Sum. Under every fixed polynomial bound on input-integer bit length, it uses $\text{poly}(n)2^{n/2}$ time and ordinary $O(2^{n/5})$ writable words of $O(n+b)$ bits, where b is the largest input bit length. Both resource bounds hold on every execution. The error is one-sided: the algorithm always rejects unsolvable instances and accepts each solvable instance with probability at least $2/3$.

1 Introduction

Given positive integers $a_1, \dots, a_n$ and a nonnegative target t , *Subset Sum* asks whether there is a set of indices $I \subseteq \{1, \dots, n\}$ such that $\sum_{i \in I} a_i = t$. Repeated values are allowed, and the empty set is a possible solution. The parameter $n \geq 2$ counts the input integers, not their total encoding length. We study the memory needed to solve this problem while retaining running time polynomial times $2^{n/2}$.

The classical meet-in-the-middle algorithm of Horowitz and Sahni [8] splits the indices in half, lists the subset sums of each half, and searches for two complementary weights. Its time and space are $O^*(2^{n/2})$, where O^* suppresses factors polynomial in the input length. Schroeppe and Shamir [10] retained this time bound with space $O^*(2^{n/4})$: four smaller stored lists suffice to generate the two large lists in sorted order. Their construction makes the distinction between generated information and stored information central to the time-space question.

Nederlof and Węgrzycki [9] reduced the space bound to $O^*(2^{0.249999n})$ with a randomized worst-case algorithm combining multiple decompositions of a solution with searches for disjoint sets of complementary weight. Belova, Chukhin, Kulikov, and Mihajlin [3] obtained $O^*(2^{0.246n})$ space at the same time bound, for input integers of magnitude $2^{O(n)}$. Polynomial-factor improvements to the running time are also known in specific word-RAM models, through the work of Chen, Jin, Randolph, and Servedio [4]. The question here is how far the space can be reduced at the half-exponential time scale.

1.1 Model and main result

We give a uniform algorithm with ordinary $O(2^{n/5})$ working space under each fixed polynomial bound on the input-integer bit length. The distinction between ordinary and polynomial-suppressed space requires us to specify the model. Put

$$b = \max(1, \lceil \log_2(1 + \max(t, a_1, \dots, a_n)) \rceil), \quad u = n + b + 2, \\ w = \lceil 4(n + b + \log_2(n + 2)) \rceil.$$

The input occupies read-only random-access memory, one integer per word. Unit-cost operations are word reads and writes, indirect addressing, comparisons, addition, subtraction, multiplication,

integer quotient and remainder with nonzero divisor, bitwise Boolean operations, and shifts. Each operation has a constant number of w -bit inputs and outputs; arithmetic exceeding these limits must be implemented and charged as multiple operations. A fresh independent uniform w -bit random word costs one operation.

Every writable register, address, counter, stack, table, seed, and retained random value counts toward working space. Old randomness can be reused only if it remains in this counted storage. The only uncharged storage is the read-only input and the fixed finite program; there is no random oracle, advice, or externally supplied table depending on n or the input. All preprocessing, input accesses, table construction, and random sampling count toward running time. Space is the maximum number of writable words simultaneously in use.

Our polynomial-bit convention fixes a positive integer c and considers inputs with $b \leq n^c$. The algorithm is not given c . Its asymptotic resource constants and threshold may depend on c , whereas correctness is required on every individual input.

Theorem 1.1. *There is a single uniform classical randomized algorithm in this model that never answers YES incorrectly and answers each YES instance correctly with probability at least $2/3$. For every fixed positive integer c , there are a constant $A_c > 0$ and positive integers k_c, N_c such that, whenever $n \geq N_c$ and $b \leq n^c$, every outcome of the algorithm’s randomness uses at most*

$$A_c n^{k_c} 2^{n/2} \quad \text{operations} \quad \text{and} \quad A_c (n + b + 2)^{15} 2^{0.199n} \quad \text{writable words.}$$

The algorithm does not depend on c .

Corollary 1.2. *For every fixed $\rho > 0.199$, the same algorithm has ordinary $O_{c,\rho}(2^{\rho n})$ word space under each fixed polynomial-bit convention, with the time and correctness guarantees of Theorem 1.1. In particular, it has $O_c(2^{n/5})$ word space.*

For fixed c and $\rho > 0.199$, the factor $(n + b + 2)^{15}$ is absorbed by $2^{(\rho - 0.199)n}$. At the endpoint 0.199 the polynomial factor remains. The resource bounds apply simultaneously on each run, including unsuccessful runs. Randomness affects the probability of finding a solution; it does not permit the algorithm to exceed its time or memory budget.

1.2 Representations and their predecessors

A representation method gives a solution many decompositions, then keeps only decompositions satisfying selected congruences. The multiplicity creates room to filter the search without losing every copy of the solution. Howgrave-Graham and Joux [7] used this idea, including recursive representations and modular filtering, for random hard-knapsack instances. Dinur, Dunkelman, Keller, and Shamir [5] developed recursive dissection tradeoffs for knapsack under randomness assumptions. Austrin, Kaski, Koivisto, and Mänttä [1] adapted that dissection framework to worst-case Subset Sum using randomized modular reductions and explicit resource controls.

For the present argument, the number of *distinct* partial sums matters as much as the number of decompositions. The residue-coverage argument of Austrin, Kaski, Koivisto, and Nederlof [2, Proposition 3.5], extended to product moduli by Nederlof and Węgrzycki [9, full version, Lemma 3.2], turns many distinct weights into many occupied residue classes. Nederlof and Węgrzycki also combined multilevel representations with weighted disjointness tests. Their construction uses separating sets that contain one choice and avoid the other, building on the separating collections of Fomin, Lokshtanov, Panolan, and Saurabh [6, Section 4.2]. We use the additional-prime filtering of Belova et al. [3, full version, Sections 1.2.3 and 4.1], together with their separator cost estimate for the required range of mask sizes [3, full version, Section 4.7]. The proofs below supply the modified enumeration, probability, and workspace arguments needed for our construction.

1.3 Two trials and the proof strategy

For a set V of input indices, let $N(V)$ be the number of distinct subset sums on V , and define its *deficiency* by

$$d(V) = |V| - \log_2 N(V).$$

A block of large deficiency has a short list of distinct weights; a block of small deficiency supplies many distinct choices for a representation. We analyze two trials according to the mean deficiency of a uniformly chosen block of a fixed size. The algorithm does not compute that mean: each round runs both trials with fresh randomness.

Compress repeated sums. The first trial chooses seven disjoint blocks and retains the distinct subset sums in each. In the high-mean case, concentration makes these lists short with high probability. Their disjoint domains allow us to forget which subset produced a retained weight. Sorted pair-sum streams combine the lists without storing their full Cartesian products. A random prime partitions the left combinations into residue classes; the trial stores one class at a time and skips it if its dictionary exceeds capacity. Success requires only the class containing a fixed solution’s left weight to fit.

Create and filter representations. The second trial uses three disjoint blocks M_0, M_L, M_R . Its two partial solutions, called *parents*, may both use indices of M_0 . Each parent is itself a join of two *children*, which may overlap on M_L on the left or M_R on the right. All remaining indices belong to disjoint background parts. Every join checks disjointness on its shared block. Each child is generated from two stored lists, giving eight *leaf lists* altogether. Within each parent, every index of M_0 belongs to exactly one leaf domain.

For a fixed solution to have many such decompositions, first choose a uniform set F of indices, negate its weights, and replace the target by $t' = t - \sum_{i \in F} a_i$, using the original weights in the sum. An original solution S becomes the uniform subset $S \triangle F$ of the signed instance, while all deficiencies stay unchanged. In the low-mean case, suitable pieces of this signed solution have many distinct sums. Splitting those pieces among the parents and children gives representations that survive nested modular filters with inverse-polynomial probability.

Separate generation work from stored records. A parent record consists of its exact weight and its set $D \subseteq M_0$ of shared indices, encoded as a membership mask. Equal weights with different masks remain different records because they can have different compatibility with the opposite parent. Generation examines raw tuples of four leaf occurrences before rejecting local overlaps and deduplicating records. Its work therefore depends on tuple counts, not only on dictionary size.

An additional independent prime subdivides the filtered parents into buckets. Different weights are unlikely to collide in the solution’s bucket. Equal weights, however, always stay together. The prescribed left-mask size bounds their number below capacity, so a left bucket can simply be skipped if it overflows. On the right, overflow instead fixes successive bits of the M_0 mask and regenerates each part. Each restriction is imposed in the leaf lists before joining them. Consequently every raw tuple belongs to at most one task at each prefix depth, including tuples later rejected for overlap. Its repeated processing is therefore charged by depth. Leaf regeneration and child-stream scans are still paid per task; bounds on bucket and distinct record counts control those task starts and the repeated matching checks.

Match disjoint masks within the budget. The final check seeks records (x, D) and (y, E) with $x + y = t'$ and $D \cap E = \emptyset$. Partition M_0 into four blocks and sample separating sets on each block. Fixing, on every block, a sampled set containing the restriction of D and avoiding the restriction of E leaves only an exact two-sum test on weights. A depth-first traversal stores only small intermediate arrays; the expected number of appearances of each record controls its work.

The success proof fixes a representation before exposing the extra prime or separating families. Because representation survival is rare, it bounds resource failure unconditionally and subtracts that bound before applying the later random choices. Dictionary capacities bound space on every run, and bounded sampling and an operation counter bound time. Independent repetitions then give Theorem 1.1.

Reading path. Section 2 establishes the counting and sorted-stream tools. Section 3 gives the complete compressed-block trial. Section 4 specifies the representation layout, filters, and bounded dictionaries, then proves the deterministic accounting for prefix refinement. Section 5 supplies the weighted-disjointness procedure. Section 6 combines representation survival, joint resource estimates, the additional prime, and the separating families; it is here that the accounting bounds receive their numerical estimates. Finally, Section 7 implements the sampling and arithmetic, enforces the budgets, and completes the amplification argument.

2 Counting and enumeration tools

Both trials use the same three facts about an arbitrary fixed input. The number of distinct sums changes little when one index is replaced; prescribed-cardinality choices have accurate entropy estimates; and a random prime rarely identifies two fixed unequal integer weights. We prove these facts first. We then give the small-state streams that turn stored lists into much larger generated lists.

All logarithms are base two unless written $\ln$. Every finite decimal constant is an exact rational. Polynomial factors below have absolute degrees independent of the input-bit promise. We use the parameters

$$\begin{aligned} u &= n + b + 2, & m &= 8\lfloor 3n/100 \rfloor, & s &= m/2, & g &= m/4, \\ C &= 2^{\lfloor .199n \rfloor}, & B &= 2^{\lfloor .19n \rfloor}, & h_* &= h(1/4), \end{aligned} \tag{1}$$

where $h(x) = -x \log x - (1 - x) \log(1 - x)$ has its continuous endpoint values. Blocks used for compression have size s ; the representation trial will use blocks of size m and their halves and quarters. The capacity C counts distinct dictionary entries, whereas B counts buffered occurrences in a stream. An entry can have many occurrences, so these capacities have different roles.

The constructions below concern the *main branch*, defined by $n \geq n_0$ and

$$u \leq 2^{\lfloor n/10^9 \rfloor}, \tag{2}$$

for one sufficiently large absolute integer n_0 fixed in the program. Outside this branch the program uses exact subset enumeration. Section 7 gives the finite implementation of this test, the repetition schedule, and the operation cap. Each eventual fixed polynomial-bit class enters the main branch.

2.1 How many distinct sums does a block have?

For a domain $V \subseteq \{1, \dots, n\}$, let $N(V)$ count the *distinct integer values* of its subset sums, and define

$$d(V) = |V| - \log N(V).$$

We call this its deficiency. In particular, $N(\emptyset) = 1$ and $d(\emptyset) = 0$. Here an actual weight is an integer sum, as opposed to a residue; masks always encode sets of indices, even when some input values agree. The next lemma applies also to signed weights, which arise later after a sign transformation.

Lemma 2.1 (Deficiency). *For arbitrary fixed weights on the n indices, deficiency is nonnegative, invariant under sign changes of individual weights, inclusion-monotone, and superadditive on disjoint domains. Replacing one index in a domain of fixed size changes its deficiency by at most one. If V is uniform among the size- v domains, $0 \leq v \leq n$, then*

$$\text{Var}(d(V)) \leq v. \quad (3)$$

Proof. There are $2^{|V|}$ subsets, so $N(V) \leq 2^{|V|}$. Adding one index multiplies the number of sums by a factor between one and two. Its addition therefore increases deficiency by a quantity in $[0, 1]$, proving inclusion monotonicity. For disjoint V, W , every sum on their union is a sum of one value from each domain, so $N(V \cup W) \leq N(V)N(W)$. Taking logarithms gives $d(V \cup W) \geq d(V) + d(W)$.

For sign invariance, let $F \subseteq V$ be the negated indices. Toggling F in a chosen subset is a bijection on all subsets of V . The new weight of a subset is the old weight of its toggle minus the old weight of F , so the set of sum values is merely translated. Finally, two equal-sized domains differing in one index are extensions of their common part. Each extension increases deficiency by a quantity in $[0, 1]$, which proves the replacement bound.

For the variance, expose a uniform ordered size- v sample without replacement. Let M_r be the conditional expectation of the final deficiency after the first r positions, $0 \leq r \leq v$. Given the first $r - 1$ positions, couple the remaining completions for two possible next indices by transposing those indices. The final domains then agree or differ in one index. Their deficiencies, and hence their conditional means, differ by at most one. Thus all possible values of M_r given the past lie in an interval of length at most one, and $|M_r - M_{r-1}| \leq 1$. Orthogonality of martingale differences now gives

$$\text{Var}(d(V)) = \sum_{r=1}^v \mathbb{E}(M_r - M_{r-1})^2 \leq v.$$

The empty-sample case is immediate. □

Consequently, for any fixed $\epsilon > 0$, a deviation of ϵn from the size-specific mean has probability $O(1/n)$, by Chebyshev's inequality. This concentration concerns a random *domain* in a fixed input; it assumes no distribution of the numerical input values.

2.2 Exact entropy estimates

Cardinality restrictions on a block reduce its subset count from 2^v to a binomial coefficient. We will need the exact exponential rate, including in a later estimate with no exponential slack.

Lemma 2.2 (Elementary entropy bounds). *For integers $0 \leq k \leq v$,*

$$\frac{2^{vh(k/v)}}{v+1} \leq \binom{v}{k} \leq 2^{vh(k/v)}.$$

For $v = k = 0$, interpret $vh(k/v)$ as zero. Entropy is concave, symmetric about $1/2$, and increasing on $[0, 1/2]$. Moreover,

$$h_* < .8114, \quad h(1/3) < 14/15, \quad h(1/6) > 5/8. \quad (4)$$

Proof. For $0 < k < v$, consider the binomial distribution with success probability k/v . The probability of exactly k successes is $\binom{v}{k} 2^{-vh(k/v)}$. It is at most one and at least $1/(v+1)$: adjacent probability ratios show that k is a mode, and there are $v+1$ possible outcomes. This gives both estimates. The cases $k=0, v$ and $v=0$ are immediate. Differentiation gives the asserted monotonicity and concavity; symmetry follows from the formula for h .

For the strict numerical inequalities, first $h_* = 2 - \frac{3}{4} \log 3$. We have $3^{12}/2^{19} > 1 + 1/74$, whose fourth power exceeds $1 + 4/74 = 39/37 > 256/243$. Hence $3^{53} > 2^{84}$ and $\log 3 > 84/53 > 1.5848$, giving $h_* < .8114$. Next, $h(1/3) = \log 3 - 2/3$ and $3^5 < 2^8$, so $h(1/3) < 8/5 - 2/3 = 14/15$. Finally, $6^2 > 2^5$ and $(6/5)^4 > 2$ imply

$$h(1/6) = \frac{1}{6} \log 6 + \frac{5}{6} \log(6/5) > \frac{1}{6} \cdot \frac{5}{2} + \frac{5}{6} \cdot \frac{1}{4} = \frac{5}{8}.$$

Thus none of the three bounds relies on floating-point optimization. $\square$

2.3 Random primes and the sampling interface

The modular filters must control collisions of actual weights, even when their bit length grows polynomially with n . We use the following elementary estimate with its full bit-length dependence.

Lemma 2.3 (Prime factors). *Let $4 \leq \ell \leq n$ be an integer. The set*

$$\mathcal{P}_\ell = \{p \text{ prime} : p < 2^\ell, 4\ell p \geq 2^\ell\}$$

has at least $2^\ell/(4\ell)$ elements. If p is uniform on this set and Δ is a fixed nonzero integer with $|\Delta| < 2^{2u}$, then

$$\Pr[p \mid \Delta] \leq 8u^2 2^{-\ell}.$$

In particular, this estimate applies to every nonzero actual-weight difference used by the two trials.

Proof. Put $X = 2^\ell$. The central binomial coefficient obeys $\binom{X}{X/2} \geq 2^X/(X+1)$. In its factorial formula, the exponent of a prime p is

$$\sum_{r \geq 1} \left(\left\lfloor \frac{X}{p^r} \right\rfloor - 2 \left\lfloor \frac{X/2}{p^r} \right\rfloor \right).$$

Each summand is zero or one, and at most $\lfloor \log_p X \rfloor$ of them are nonzero. Thus each prime contributes a factor at most X to the binomial coefficient. If $\pi(X)$ is the number of primes at most X , taking logarithms gives

$$\ell \pi(X) \geq X - \log(X+1) \geq X/2 \quad (X \geq 16).$$

Remove primes below $X/(4\ell)$. There are at most $X/(4\ell)$ integers in that range, so at least $X/(4\ell)$ primes remain. The upper endpoint X is composite, proving the cardinality bound.

An integer Δ with $0 < |\Delta| < 2^{2u}$ has at most $2u$ distinct prime divisors: the product of r distinct primes is at least 2^r . Dividing by the number of available primes and using $\ell \leq n \leq u$ gives the displayed collision bound. Actual weights and their relevant differences have magnitude $O(n2^b)$; the constructions sum each indexed weight at most twice, even before rejecting an overlapping tuple. They therefore meet $|\Delta| < 2^{2u}$ for sufficiently large main-branch inputs. $\square$

We analyze uniform independent draws, conditional on each specified finite range. The implementation uses at most n^3 attempts for each draw and fails the current trial if they all fail. The precise interface, proved in Lemma 7.1, is as follows. A draw from $\{0, \dots, D-1\}$ has the uniform

law on success and couples to an ideal draw with failure probability at most 2^{-n^3} ; $D = 1$ needs no randomness. A draw from $\mathcal{P}_\ell$ has its uniform law on success and coupling failure at most $e^{-n^2/4}$. These conditional bounds remain valid for ranges determined by past choices. All requested prime parameters satisfy $\ell \leq .258n$; bounded trial division therefore costs at most $\text{poly}(n)2^{.129n}$ per prime search, including all its attempts. The final section proves the finite-word implementation and the aggregate coupling estimate. The trials below use no unbounded sampling operation in their resource guarantees.

2.4 Sorted modular pair streams

A stream will enumerate a Cartesian product while retaining only its smaller input arrays. An item consists of its actual, possibly signed, weight and a constant number of index masks. Write $\omega(a)$ for the actual integer weight of an item a . Repeated occurrences remain separate even if all their fields agree. Each item occupies constantly many words; array indices have $O(n)$ bits. We assume that weights, moduli, keys, and indices fit in the stated constant-word representation. Section 7 verifies these conditions for all uses below.

This is the sorted pair-sum method associated with Schroeppe and Shamir [10]; see also the explicit four-list description in Nederlof and Węgrzycki [9, full version, Lemmas 3.3 and A.1]. We give the modular ordering and stopped-execution argument locally, since those details determine the cost of regenerating a dictionary. A generator’s mutable state identifies its next occurrence and can be copied while sharing its immutable input arrays.

Lemma 2.4 (Filtered streams and tie joins). *Let A_1, A_2 be stored constant-word item arrays whose indices have $O(n)$ bits. Let $p \mid P$ be positive integers, and let c, v be specified integer residues. Assume that the data occupy constantly many words each and that arithmetic, comparisons, and indexed access on them take $O(1)$ word operations in the stated model. The pairs $(a, b) \in A_1 \times A_2$ whose actual weights sum to $c \pmod{p}$ can be streamed in nondecreasing order of either*

$$(\omega(a) + \omega(b)) \bmod P \quad \text{or} \quad (v - \omega(a) - \omega(b)) \bmod P,$$

where all residues are normalized to $\{0, \dots, P-1\}$. Initialization costs $O(n(|A_1| + |A_2| + 1))$ word operations, each subsequent advance costs $O(n)$, and the state together with the input arrays uses $O(|A_1| + |A_2| + 1)$ words.

Consider two such streams of lengths N_1, N_2 , with combined states and arrays of at most S words. If their equal-key Cartesian products contain Z pairs in all, they can report all these pairs using $O(S + B)$ words and, apart from initialization and the work performed on reported pairs,

$$O\left(B + n(N_1 + N_2 + Z) + \frac{S}{B}Z\right) \tag{5}$$

operations. Here $B \geq 1$ is an integer buffer capacity. In particular, if $S \leq \text{poly}(n)B$, the bound is $\text{poly}(n)(B + N_1 + N_2 + Z)$. The same upper bounds hold for an execution stopped partway, with Z still counting all potential matches. No coprimality assumption is needed.

Proof. First sort A_2 lexicographically by its residue modulo p and then its residue modulo P . For a fixed $a \in A_1$, binary search finds the contiguous slice with residue $c - \omega(a) \pmod{p}$. This slice contains exactly the eligible partners of a , and is ordered by their P -residues, including repetitions. Omit the row if the slice is empty.

For a direct key, put $r = \omega(a) \bmod P$. Start at the first entry of the slice whose residue is at least $P - r$; if there is none, start at the beginning. Traverse forward cyclically for exactly the

slice length. The entries at or above the pivot give the wrapped values $r + (\omega(b) \bmod P) - P$ first, followed by the unwrapped values $r + (\omega(b) \bmod P)$. Both portions are nondecreasing and the former values are smaller than the latter. This includes $r = 0$, when the pivot search simply returns the beginning.

For a complementary key put $K = (v - \omega(a)) \bmod P$. Start at the last entry whose residue is at most K ; if there is none, start at the last entry of the slice. Traverse in reverse cyclic order for exactly one slice length. Residues at most K produce the values $K - (\omega(b) \bmod P)$ in nondecreasing order. The residues above K follow and produce the larger values $K - (\omega(b) \bmod P) + P$, also in nondecreasing order. Choosing the first pivot in the direct case and the last pivot in the complementary case preserves every repeated residue.

Merge the rows with a min-heap having at most one current pair per item of A_1 . A row stores its slice endpoints, current position, and remaining length; a heap entry has constantly many words. Comparison sorting, binary searches, and heap operations have depth $O(n)$ because array indices have $O(n)$ bits. This proves the initialization, advance, and state bounds. The construction permits repeated prime factors and other common factors. Empty input arrays give an empty stream.

To join two streams, advance past unequal keys until an equal key is found. For that key, read up to B second-stream items into a reusable buffer. If this exhausts the second tie group, pair its buffered items with each consecutive first-stream item of the key. No generator state is copied in this case. Reuse the buffer by its current length, without clearing B positions for a short group.

If the second tie group is longer, save the mutable second-generator state immediately after the first B items. The snapshot includes its current lookahead item, heap, row cursors, and control state, but shares its immutable arrays. For each first-stream item of the key, report its pairs with the buffer, restore the saved state when needed, and replay the remaining suffix. After the final replay the second stream is already beyond the tie group.

A save or restore costs $O(S)$. It is made only for a first item having at least B potential partners; at most a constant number of such operations per first item is required. Their total cost is therefore $O(SZ/B)$. Replayed advances are charged to matching pairs; other advances are charged to the original stream lengths. The $O(n)$ cost per advance and the optional $O(B)$ initial buffer allocation give (5). If processing stops immediately after a snapshot or restore, the same charge remains valid because Z counts potential pairs, including pairs not reached. One snapshot, the buffer, and the two generators give the space bound. $\square$

An actual-weight stream needs no modular pivots. Sort the second array by actual weight, add each first-array weight to that sorted row, and heap-merge the rows. Reading this ordered stream while remembering only the last weight counts its distinct values without storing the Cartesian product. This will let the representation trial measure a block's deficiency exactly from two small half-block lists.

2.5 Bounded dictionaries

Streaming saves space only if we also control what is retained. For deterministic deduplication, use a binary trie on a fixed-width signed actual weight, followed by the relevant mask bits when a mask is part of the key. Give zero its canonical nonnegative sign. In our applications the complete key has $O(u)$ bits. Searching or inserting it takes $O(u)$ operations and introduces at most $O(u)$ nodes, with constantly many words per node.

Keep a separate append-only array of the distinct entries if subsequent array access is needed. A duplicate changes neither the array length nor its mathematical content. On finding a new key that would increase the array beyond capacity C , report overflow. Even if nodes for this first rejected key

have been allocated, the trie has at most $O(u(C + 1))$ nodes. All nodes, addresses, and appended records count as workspace. A weight-only dictionary is appropriate for disjoint compression blocks; a weight-mask dictionary will be required when compatibility depends on shared indices.

For the concrete implementations below, raw list generation can be done in $O(n^2)$ operations per item, sorting and heap work in $O(n)$ per item or advance, and dictionary operations in $O(u)$ per key. Whenever the combined stream storage is $O(B)$, the term depending on Z in (5) is $O(nZ)$, before the separately charged processing of those matches. The buffer and stream-length terms remain separate. These explicit degrees will be used when the task costs are bounded. The argument does not assign a cost merely from the number of distinct entries eventually retained.

3 Compressing repeated sums

The first trial applies when random blocks have many repeated subset sums. It retains only the distinct weights of seven disjoint blocks, uses those small lists to generate four larger groups, and joins the groups one residue class at a time. Because the domains are disjoint, we can forget which subset produced a retained weight. A hard capacity bounds memory; success requires only the residue class of one fixed solution to fit.

This trial uses the classical stored-list and generated-stream viewpoint of Schroeppe and Shamir [10]. Its modular regeneration and overflow rule also belong to the dissection approach: Dinur, Dunkelman, Keller, and Shamir [5] developed the earlier framework under randomness assumptions, and Austrin, Kaski, Koivisto, and Määttä [1] adapted dissection to worst-case Subset Sum. The trial and its bounds here are proved directly, including the dependence on the input bit length.

3.1 Seven compacted blocks and four groups

Take the first $7s$ indices of a uniform random permutation and divide them into seven consecutive blocks $K_1, \dots, K_7$ of size s . For each i , enumerate and sort all its subset sums, then compact repeated weights into a list D_i of distinct actual weights. Continue only if every list has at most 2^a entries, where

$$a = \lfloor s - .0075n \rfloor. \quad (6)$$

This preliminary enumeration stores at most 2^s weights at a time. The seven retained lists are stored separately.

Of the remaining indices, give

$$r_R = \lfloor .05n \rfloor \quad \text{to the right and} \quad r_L = n - 7s - r_R \quad \text{to the left.}$$

Choose these domains in their permutation order. Split the right raw domain into V_{R1}, V_{R2} of sizes r_{R1}, r_{R2} differing by at most one, and call the left raw domain V_L . All these sizes are nonnegative on sufficiently large main-branch inputs.

For a domain V , write $\mathcal{S}(V)$ for its full list of indexed subset-sum occurrences, retaining repetitions. For two lists A, D write $A \oplus D$ for the list with one occurrence of weight $x + y$ for each pair $(x, y) \in A \times D$. Table 1 specifies the four generated groups by their two stored lists. A group stream enumerates the pair sums of the two lists in its row. The eight stored lists use separate subdomains, and the four group domains partition the input indices.

Sample q uniformly from $\mathcal{P}_\ell$ with $\ell = \lfloor .249n \rfloor$. For each integer $z \in \{0, \dots, q - 1\}$, proceed as follows.

Group	Index domain	First stored list	Second stored list
$\mathcal{L}_1$	$K_5 \cup K_6$	D_5	D_6
$\mathcal{L}_2$	$K_7 \cup V_L$	D_7	$\mathcal{S}(V_L)$
$\mathcal{R}_1$	$K_1 \cup K_2 \cup V_{R1}$	D_1	$D_2 \oplus \mathcal{S}(V_{R1})$
$\mathcal{R}_2$	$K_3 \cup K_4 \cup V_{R2}$	D_3	$D_4 \oplus \mathcal{S}(V_{R2})$

Table 1: The seven compacted block lists generate four groups. Each row is streamed from its two stored lists. The raw and product lists retain occurrences, while each D_i retains only distinct weights. No two rows share an input index.

1. Generate $\mathcal{L}_1$ by its direct key modulo q and $\mathcal{L}_2$ by its complementary key $z - \text{weight}$ modulo q . Join equal keys using Lemma 2.4, with $p = 1$ and $P = q$. Insert each resulting actual total weight into a weight-only dictionary of capacity C .
2. If this dictionary overflows, discard it and move to the next z . Otherwise retain its completed list of distinct weights. An empty completed dictionary may also be skipped.
3. Generate and join $\mathcal{R}_1, \mathcal{R}_2$ in the same way, with target residue $t - z \pmod{q}$. For every resulting actual right weight y , search for $t - y$ in the left dictionary. A hit returns YES. Otherwise free the dictionary and continue the residue loop.

A sampling failure also fails the trial. The bounded samplers and operation counter are those of Section 7.

Every retained block weight has a subset witness in its block. The four group domains are disjoint, so any choices witnessing an exact left–right weight match combine into an indexed subset of weight t . Thus the trial has one-sided correctness. It need not retain witnesses or output them: its required output is the decision YES.

3.2 Work, space, and success

Let μ be the mean of $d(V)$ when V is uniform among the size- s domains of the given input. The algorithm never computes this mean. It runs this trial and the representation trial in the same round; the following hypothesis identifies the case in which this first trial supplies success.

Lemma 3.1 (Compressed-block trial). *For sufficiently large main-branch inputs, the compressed-block trial uses at most $u^{15}2^{n/2}$ word operations on every random outcome, including bounded sampling, and uses $O(u(C+1) + B + 2^s + 2^{14n})$ writable words. Every returned YES is correct. If the input is a YES instance and*

$$\mu \geq .00765n, \tag{7}$$

its success probability is bounded below by an absolute positive constant.

Proof. First bound the stored and generated lists deterministically, assuming the seven size tests pass. To retain the effects of floors, put

$$\delta = .12n - s \in [0, 4), \quad \epsilon = .05n - r_R \in [0, 1).$$

Then $r_L = .11n + 7\delta + \epsilon$ and $a \leq .1125n - \delta$. Each stored product list on the right has at most

$$2^{a + \lceil r_R/2 \rceil} \leq 2^{.1375n + 1/2}$$

occurrences. The left raw list has at most $2^{.11n+29}$ occurrences, and each D_i has at most $2^{.1125n}$ entries. Hence every stored sublist in Table 1 has size at most $2^{.14n}$ for sufficiently large n .

Let $N_{L1}, N_{L2}, N_{R1}, N_{R2}$ denote the lengths of the four group streams, counting occurrences. Their direct product bounds are

$$\begin{aligned} N_{L1} &\leq 2^{2a} \leq 2^{.225n}, & N_{L2} &\leq 2^{a+r_L} \leq 2^{.2225n+25}, \\ N_{Ri} &\leq 2^{2a+r_{Ri}} \leq 2^{.25n+1/2} & (i = 1, 2). \end{aligned}$$

More importantly, the products that count complete combinations obey

$$N_{R1}N_{R2} \leq 2^{4a+r_R} \leq 2^{.5n-4\delta-\epsilon} \leq 2^{n/2}, \quad (8)$$

$$N_{L1}N_{L2} \leq 2^{3a+r_L} \leq 2^{.4475n+4\delta+\epsilon} < 2^{.4475n+17}. \quad (9)$$

These count all combinations, not just distinct resulting weights.

Each complete left combination has exactly one total residue modulo q , and each right combination has exactly one complementary residue $t - \text{weight}$. Across the whole residue loop, the number of potential Cartesian matches is consequently at most $N_{L1}N_{L2} + N_{R1}N_{R2}$. Overflow may stop a join partway, but Lemma 2.4 still charges it to this same potential count. Repeated weights therefore cause neither an uncounted replay cost nor a loss of a combination.

Here is a concrete operation accounting. Generating an item, including its weight, uses at most $O(n^2)$ operations; comparison sorting and heap operations use $O(n)$ per item or advance; trie operations use $O(u)$ per key. The stream states for a group have $O(2^{.14n})$ words and are therefore $O(B)$ for large n . Thus the match-dependent term in (5) has $O(n)$ overhead per potential match; its buffer and stream-length terms are charged separately below. We may even initialize $O(u(C+1))$ trie space afresh at each residue. If A is the total number of stored sublist entries and $N = N_{L1} + N_{L2} + N_{R1} + N_{R2}$, all nonsampling work is bounded by

$$O\left(u^3 [2^s + A + q(N + B + u(C+1) + 1) + N_{L1}N_{L2} + N_{R1}N_{R2}]\right). \quad (10)$$

The 2^s term pays the seven initial enumerations and compactions, the terms multiplied by q pay regeneration and initialization, and the final two terms pay every potential match and dictionary action. The bound also covers a failed size test.

Since $q < 2^{.249n}$, the largest regenerated-stream exponent is $.249 + .25 = .499$. The buffer and dictionary initialization exponents are $.439$ and $.448$, respectively. Equations (8)–(9) bound the remaining products, so (10) is $O(u^4 2^{n/2})$. A permutation uses at most $O(n^4)$ bounded range attempts and elementary operations, and the one prime search costs at most $\text{poly}(n)2^{.129n}$ with the fixed elementary trial-division implementation of Lemma 7.1. In particular it is $O(n^4 2^{.129n})$. These smaller terms leave the total below $u^{15} 2^{n/2}$ for sufficiently large n . This deterministic bound is below the trial cap; it does not assume any favorable bucket sizes or random events.

For space, the initial enumerations use $O(2^s)$ words, the stored sublists and stream states use $O(2^{.14n})$, tie replay uses $O(B)$, and the one retained dictionary uses $O(u(C+1))$. Sorting needs only linear auxiliary arrays. The permutation and loop state are absorbed by these bounds. In particular, overflow enforces the space bound on every outcome.

It remains to show that one solution bucket fits in the high-mean case. Each random K_i is marginally a uniform size- s domain; the seven blocks need not be independent. Lemma 2.1, Chebyshev's inequality, and a union bound give

$$\Pr[d(K_i) \geq .00755n \text{ for all } i] = 1 - O(1/n)$$

under (7). These inequalities imply $N(K_i) \leq 2^{s-.00755n} \leq 2^a$ for sufficiently large n , so all tests pass, including their floors.

Fix an original solution before sampling the partition. Conditional on any passing partition, let x be its left weight. There are fewer than $2^{.4475n+17}$ distinct left weights in all. For each different one x' , Lemma 2.3 and $\ell = \lfloor .249n \rfloor$ give

$$\Pr[x' \equiv x \pmod{q}] \leq 8u^2 2^{-\ell} \leq u^3 2^{-.249n}$$

for large n . By linearity of expectation, the number Y of other distinct weights in x 's residue class satisfies

$$\mathbb{E}Y \leq u^3 2^{.1985n+17}.$$

Because $C \geq 2^{.199n-1}$ and $\log u \leq n/10^9$,

$$\frac{\mathbb{E}Y}{C} \leq 2^{(3/10^9-.0005)n+18} = o(1).$$

Markov's inequality implies $Y + 1 \leq C$ with probability tending to one. The complete dictionary in that residue then retains x , and the right solution weight produces a hit. Earlier residues can only produce an earlier correct YES or be skipped.

This proves success with probability $1 - o(1)$ in the ideal experiment. The permutation and prime draws have negligible total bounded-sampler coupling loss by Lemma 7.1; the deterministic work estimate prevents the operation counter from aborting a completed trial for sufficiently large n . Thus an absolute positive success probability remains for the implemented trial. $\square$

Compression has now handled the case in which random blocks have few distinct sums. When $\mu < .00765n$, the next trial instead uses the many distinct sums to preserve a representation of a fixed solution through several modular filters.

4 Representations and bounded parent dictionaries

The second trial uses shared index sets to give a solution many representations. It must generate those representations without storing all of them. We first specify the shared sets and the leaf lists, then the modular filters and a finite schedule of bounded dictionary tasks. The last part of this section proves deterministic bounds for that schedule. Lemma 6.1 will show that the filters retain a representation; Section 6.2 will bound the work needed to bring it to a completed check.

The trial runs alongside the compression trial in every round. It does not test the mean deficiency μ or decide which trial should apply. The representation method and modular filtering have antecedents in Howgrave-Graham and Joux [7]; the multilevel worst-case setting and its weighted disjointness step are developed by Nederlof and Węgrzycki [9]. The construction below specifies the shared indices, exact integer parameters, and bounded recomputation needed for the present resource bounds.

4.1 Shared mixers and leaf lists

This trial gives each parent two children and each child two leaves. The leaves are stored; children and parents are generated from them. We first describe their index domains and prescribed cardinalities, then balance the unconstrained background parts so that every leaf is small enough to store. The shared blocks, which we call *mixers*, create multiple ways to represent a solution and require explicit disjointness tests.

Select three disjoint size- m sets uniformly from a random permutation. For each, compute its number of distinct actual subset sums exactly: store the subset-sum lists of two size- s halves and count distinct weights in their actual-weight sorted pair stream. Name a mixer with the largest such number M_0 , and name the other two M_L, M_R in a deterministic order, breaking ties deterministically. Put

$$N_i = N(M_i), \quad d_i = d(M_i), \quad i \in \{0, L, R\}.$$

Continue only if

$$\max_i N_i \leq (\min_i N_i) 2^{\lfloor .0005n \rfloor}. \quad (11)$$

Thus $d_0 \leq d_L, d_R$ and the deficiency spread is at most $.0005n$.

Choose a uniform subset F of all input indices, negate its weights, and replace the target by

$$t' = t - \sum_{f \in F} a_f, \quad (12)$$

where the sum uses the original weights. In the rest of this trial all weights are signed after this change. The signed weight of any subset I' is the original weight of $I' \triangle F$ minus the original weight of F . Thus the transformed instance is equivalent to the original, and Lemma 2.1 makes all N_i, d_i unchanged.

Independently guess k_i uniformly from $\{\lceil m/6 \rceil, \dots, m/4\}$, for $i = 0, L, R$, and put $\lambda_i = k_i/m$. At the top join, the left and right masks on M_0 will have sizes k_0 and $s - k_0$. Within parent $j \in \{L, R\}$, its two children will have masks on M_j of sizes k_j and $s - k_j$. Disjointness of these masks is checked at the corresponding join. The left parent never uses M_R , and the right parent never uses M_L .

Independently choose a uniform labeled partition

$$M_0 = T_1 \dot{\cup} T_2 \dot{\cup} T_3 \dot{\cup} T_4, \quad |T_a| = g.$$

Choose integers $c_1, \dots, c_4$ as equally as possible with sum k_0 , assigning the remainder in a fixed order. The left count in T_a is c_a and the right count is $g/2 - c_a$. In each parent, child 1 uses T_1, T_2 and child 2 uses T_3, T_4 , one block in each of its two leaves. In particular, within a parent the top-mixer indices in its children are disjoint.

Independently for $j = L, R$, partition M_j uniformly into two labeled blocks Q_{j1}, Q_{j2} of size s . Prescribe child-1 counts a_{j1}, a_{j2} as equally as possible with sum k_j . The corresponding child-2 counts are $s/2 - a_{j1}, s/2 - a_{j2}$. Both children use the same two blocks of M_j , one in each leaf; their possible overlap is exactly what the join of the two children into their parent checks. Figure 1 shows both parents and all eight leaves. Each leaf also receives its own disjoint background part, defined next. A chosen subset of a top block is recorded in the M_0 mask, and a chosen subset of a local block is recorded in the M_j mask.

Balancing the stored lists

There remain $r = n - 3m$ background indices, whose selections have no cardinality constraints. The prescribed mixer counts may make one parent's lists larger than the other's, or one child's lists larger than its sibling's. We compensate by assigning more background indices to the smaller side, first between parents and then between children. The program makes these allocations with integer binomial logarithms:

$$\begin{aligned} b_{0L} &= \left\lfloor \log \binom{m}{k_0} \right\rfloor, & b_{0R} &= \left\lfloor \log \binom{m}{s - k_0} \right\rfloor, \\ b_{j1} &= \left\lfloor \log \binom{m}{k_j} \right\rfloor, & b_{j2} &= \left\lfloor \log \binom{m}{s - k_j} \right\rfloor \quad (j = L, R). \end{aligned}$$

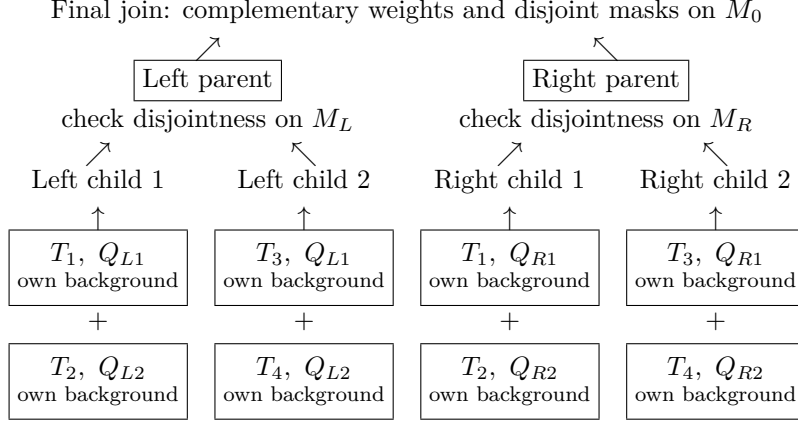


Figure 1: The eight stored leaf lists and their joins, read from bottom to top. The two boxed leaves in each column generate one child by pair-sum streaming. Each T_a occurs once on each parent side; each Q_{jr} occurs in both children of side j . The eight background parts are pairwise disjoint and avoid all three mixers. Local joins reject overlap on M_L or M_R ; the final join rejects overlap on M_0 . Boxes label allowed domains; repeated labels mark shared indices.

Set

$$r_L = \left\lfloor \frac{r + b_{0R} - b_{0L}}{2} \right\rfloor, \quad r_R = r - r_L,$$

$$r_{j1} = \left\lfloor \frac{r_j + b_{j2} - b_{j1}}{2} \right\rfloor, \quad r_{j2} = r_j - r_{j1}.$$

Thus $r_L + r_R = r$ and $r_{j1} + r_{j2} = r_j$ exactly. Split each child's background count as equally as possible between its two leaves, using any fixed order on the background indices.

To verify feasibility and list sizes, write

$$H_{0L} = h(\lambda_0), \quad H_{0R} = h(1/2 - \lambda_0), \quad \bar{H}_0 = (H_{0L} + H_{0R})/2,$$

$$H_{j1} = h(\lambda_j), \quad H_{j2} = h(1/2 - \lambda_j), \quad \bar{H}_j = (H_{j1} + H_{j2})/2 \quad (j = L, R).$$

Concavity gives $\bar{H}_0, \bar{H}_L, \bar{H}_R \leq h_*$. Lemma 2.2 translates the integer assignments into

$$r_j = r/2 + (\bar{H}_0 - H_{0j})m + O(\log n),$$

$$r_{ji} = r_j/2 + (\bar{H}_j - H_{ji})m + O(\log n). \tag{13}$$

These identities express the cancellation we need: the larger a prescribed mixer family is, the fewer unconstrained background indices its leaf receives.

The assignments are feasible for sufficiently large n . In every entropy pair the arguments lie in $[1/6, 1/3]$, so its difference in absolute value is at most $1 - h(1/6) < 3/8$. Consequently

$$r_j \geq .14n - \frac{3m}{16} - O(\log n) \geq .095n - O(\log n), \quad r_{ji} \geq \frac{.095n - .09n}{2} - O(\log n) > 0.$$

All prescribed mixer counts are feasible as well: they are rounded versions of fractions in $[1/6, 1/3]$ of their block sizes. The program may simply fail the trial if any integer feasibility test fails.

A leaf list has one occurrence for each choice with its prescribed counts in its top and local mixer blocks and an arbitrary subset of its background part. An occurrence retains its actual weight and mixer masks. Enumerate the fixed-cardinality subsets directly, for example in lexicographic order on index combinations, rather than scanning all unconstrained masks. This takes polynomial

overhead per generated item. Raw counts retain occurrence multiplicity. A raw child pair chooses one occurrence from each of its two leaves, and a raw four-leaf tuple chooses one from each of a parent's four leaves. Its weight is the sum of the occurrence weights, even when local-mixer masks overlap. The balanced background allocation makes the four leaf sizes on each side comparable. Define their common exponential scale by

$$\beta_j = r/4 + \overline{H}_0 m/2 + \overline{H}_j m \leq n/4 + (-.75 + 1.5h_*)m. \quad (14)$$

Every leaf on side j and each two-leaf child satisfy

$$\#\{\text{leaf items}\} \leq u^{10} 2^{\beta_j/2}, \quad \#\{\text{raw child pairs}\} \leq u^{20} 2^{\beta_j}. \quad (15)$$

Indeed, before substituting (13), the logarithm of the leaf size is at most

$$H_{0j}m/4 + H_{ji}m/2 + r_{ji}/2 + O(1).$$

The binomial logarithm errors in each background allocation are at most $\log(n+1) + 1$, and a rounded prescribed block count changes its entropy upper exponent by $O(1)$: for large n the relevant fractions lie in $[1/8, 3/8]$, where $|h'| < 3$. More explicitly, the error in r_j from its ideal value is at most $3 \log u$, and that in r_{ji} after both ideal substitutions is at most $5 \log u$. Substitution gives $\beta_j/2 + O(\log n)$ with ample room in the factor u^{10} for large n . Finally $h_* < .8114$ and $m \leq .24n$ imply

$$\beta_j/2 \leq .181052n.$$

Since $\log u \leq n/10^9$, every raw leaf list has fewer than $2^{.185n}$ items for sufficiently large main-branch inputs.

4.2 Nested modular filters

The first filtering level constrains each child's weight; the second constrains the sum of the two children. Their moduli are nested so that once a parent and its first child pass, the required congruence for its second child follows. An additional independent prime q subdivides the parent residue classes into buckets, following the additional-prime filtering approach of Belova, Chukhin, Kulikov, and Mihajlin [3, full version, Sections 1.2.3 and 4.1]. The low filters reduce raw generation; the extra prime controls the distinct entries in the solution's bucket.

For analysis define the real quantities

$$\begin{aligned} \tau &= .0078n, & f_i &= \min(\tau, d_i/2), & L_i &= s - f_i, \\ f_{\max} &= \max_i f_i, & \xi &= n/4 + .032m - f_{\max}. \end{aligned} \quad (16)$$

The program uses the integer parameters

$$\ell_i = \lfloor L_i \rfloor = \max \left\{ \lfloor s - \tau \rfloor, \left\lfloor \frac{\log N_i}{2} \right\rfloor \right\},$$

computed by bit lengths; flooring $\log N_i$ first gives the same second term. Since d_0 is smallest, L_0 is largest. Sort the three ℓ_i in nondecreasing order. For each increment from the preceding value, taking the initial preceding value to be zero, independently sample a prime with that increment as parameter if the increment is at least four; otherwise use the factor one. At each sorted index let p_i be the cumulative product. Tied parameters give the same product. In particular,

$$p_L \mid p_0, \quad p_R \mid p_0.$$

For every fixed nonzero relevant actual-weight difference Δ , conditional on these log parameters, the construction satisfies

$$\frac{2^{L_i}}{u^5} \leq p_i \leq 2^{L_i}, \quad \Pr[p_i \mid \Delta] \leq u^{10} 2^{-L_i} \quad (i = 0, L, R). \quad (17)$$

For the size estimate there are at most three sampled factors; the discarded increments total at most nine. The prime lower ranges and the floor therefore lose a factor of at most $2^{10}(4n)^3$ relative to 2^{L_i} , which is at most u^5 for large n . For divisibility, Δ has at most $2u$ distinct prime divisors. A factor with parameter ℓ divides it with probability at most $8u^2 2^{-\ell}$, by Lemma 2.3. All sampled factors are independent. Divisibility by their product requires divisibility by each sampled factor, so multiplication of these bounds, with the same discarded-increment and floor losses, proves (17). This argument does not require the sampled primes to be distinct.

Independently sample one more prime q , with parameter

$$\ell_q = \left\lfloor n/4 + .032m - s + \min_i \ell_i \right\rfloor.$$

Then $\xi - 2 < \ell_q \leq \xi$ and $\xi \leq .25768n$; in particular ℓ_q is positive and linear in n . The same prime estimates give

$$2^\xi / u^2 \leq q \leq 2^\xi, \quad \Pr[q \mid \Delta] \leq u^3 2^{-\xi} \quad (18)$$

for each fixed nonzero relevant difference. Write $P = p_0 q$. Choose independent uniform residues $\rho_0 \pmod{p_0}$ and $\rho_j \pmod{p_j}$ for $j = L, R$, independently also of q . Their desired parent low bases are

$$w_L = \rho_0, \quad w_R = t' - \rho_0.$$

For side j , require child 1 to have weight $\rho_j \pmod{p_j}$ and child 2 to have weight $w_j - \rho_j \pmod{p_j}$. The independent draw of q may be deferred until after all these choices when analyzing them.

4.3 Parent dictionaries and prefix recomputation

A parent dictionary stores records (x, D) : the exact signed integer weight x and the indexed top mask $D \subseteq M_0$. Its key is both fields, so equal weights alone do not cause deduplication. A parent task consists of a side, a target residue modulo P , and a possibly empty prefix of the M_0 mask in a fixed index order. Left tasks use the empty prefix; right tasks may use a longer prefix. The task regenerates its four leaves, joins its two child streams, and either completes its dictionary or reports overflow.

The final check, specified in Lemma 5.1, tests whether two parent arrays contain disjoint top masks whose actual weights add to t' . Its four random separating families are sampled once before the following loop, with the distribution specified in Section 5. Conditional on the layout, they use fresh randomness independent of every parent-generation choice, including the moduli, low residues, and extra prime, and are reused for all checks. Their values do not affect which parent tasks are generated or how those tasks process entries; a check can only stop the trial with YES.

For $z = 0, \dots, q - 1$, set

$$v_L = \rho_0 + p_0 z, \quad v_R = (t' - v_L) \pmod{P},$$

with normalized residues. On each side these range exactly once over the P -classes with the required parent low base. Since $p_j \mid p_0$, the child low requirements do not change with z .

First perform the left task. Its two child streams are obtained from their leaf pairs by Lemma 2.4, with modulus P and low modulus p_L . Stream the first child by its weight modulo P , and the

second by v_L minus its weight modulo P . Join equal keys. For each matched pair, check that its two M_L masks are disjoint. If so, retain just the actual total weight and the combined M_0 mask, deduplicating these pairs in a dictionary of capacity C . If a new distinct pair would overflow the dictionary, abandon this z . Also skip z if the completed dictionary is empty.

Otherwise retain the completed left array and perform the analogous right task with residue v_R , initially without an additional top-mask restriction. If its completed dictionary has between one and C distinct entries, apply the final check to that right array and the stored left array. If it is empty, do nothing. If it overflows, discard the task's workspace and replace the task by two right tasks that fix the next bit of its M_0 mask to zero and one, respectively. Process these tasks depth first, in a fixed order of the top-mixer bits. An overflowing task whose prefix already specifies the entire mask is simply dropped. Reuse or free the workspace after completing or discarding each task, and stop at the first YES.

In every prefix task, remove incompatible items from each leaf array *before* generating its streams. Test in that leaf exactly the prescribed prefix bits belonging to its own top block. Regenerating and filtering its raw leaf list from scratch is permitted. As each top block occurs in exactly one leaf of a parent, no raw four-leaf tuple with an incompatible top mask is generated in that task. This remains true even for tuples rejected later because their local-mixer masks overlap. It is this early restriction that permits the later charging of repeated four-leaf tuple processing by depth rather than by number of tasks.

Lemma 4.1. *Every retained parent entry has a subset witness on its parent's indices. Deduplicating entries by actual weight and top mask preserves the existence of every valid final match. Every YES returned by either trial is correct.*

Proof. Within a parent the children share only M_j , and their disjointness there has been checked. Their top blocks and background parts are disjoint. Thus a retained entry has a subset witness. The only indices shared by the left and right parents are in M_0 . Once their top masks are fixed, the choice among witnesses of a given actual weight has no effect on compatibility with the other parent. Hence deduplication loses no valid match. A final check returning YES has disjoint top masks and actual total weight t' , so its two witnesses give a signed solution, and (12) gives an original solution. The compressed-block trial was already seen to have the same one-sided correctness property. $\square$

4.4 Three bounds for prefix recomputation

The schedule has two different sources of work. Starting a task regenerates leaves and initializes streams; processing its matches examines raw four-leaf occurrences. A small dictionary can have many raw representations, including invalid local overlaps. We therefore use distinct entries to count task starts and raw occurrences to count match processing. A third count measures how often retained arrays are passed to the final check.

What the trial counts. Fix the mixers, signs, cardinality guesses, layout, low moduli, and low residues, without fixing the extra prime q or the separating families. The following three finite counts are already determined at this stage:

- G is the sum, over the four children, of the numbers of raw leaf pairs passing that child's low filter, before any mask-prefix restriction.
- $\mathcal{T}_j$ contains all raw four-leaf occurrences on side j passing the parent low filter modulo p_0 and the first-child filter modulo p_j . The second-child filter follows from $p_j \mid p_0$. Put $J_j = |\mathcal{T}_j|$, retaining every repetition and every local overlap.

- $\mathcal{U}_j$ contains all distinct valid (actual weight, top mask) entries satisfying the parent low filter, ignoring child filters. Here validity means that the entry has a subset witness on side j 's indices with the prescribed top-block counts. This ambient set may contain entries not generated by the child lists. Put $U_j = |\mathcal{U}_j|$.

After q is chosen, assign each occurrence and entry its unique bucket using the enumerated $P = p_0 q$ classes. On the right the bucket label is the value of z whose class is $(t' - \rho_0 - p_0 z) \bmod P$. These labels depend on q , but the three counts do not. Equal entry keys have equal actual weights and hence the same bucket. No coprimality of p_0 and q is needed.

The next lemma separates these deterministic charges from all probability estimates. The schedule is the one specified in Section 4.3: left overflow skips a bucket, whereas right overflow refines a mask prefix. Only for counting, continue this schedule after any YES answer and ignore the operation cap. This finite hypothetical schedule need not be stored.

Lemma 4.2 (Prefix accounting). *Fix positive integers C, q and a nonnegative integer m . On each side $j \in \{L, R\}$, let $\mathcal{T}_j$ be a finite collection of raw tuple occurrences. Each occurrence has a bucket in $\{0, \dots, q-1\}$ and a top mask in $\{0, 1\}^m$. A valid occurrence produces an entry consisting of its exact weight and top mask. Assume that occurrences producing the same entry have the same bucket. Let $\mathcal{U}_j$ be a finite ambient set containing all distinct valid entries, with bucket and mask labels extending those of the generated entries. Write*

$$J_j = |\mathcal{T}_j|, \quad U_j = |\mathcal{U}_j|.$$

A task for bucket z and prefix σ generates exactly the raw occurrences with these labels, unless it stops early, and reports overflow at the $(C+1)$ st distinct valid entry. Suppose a task with Z potential occurrences costs at most $H + KZ$, even if stopped partway, for fixed nonnegative H, K .

Apply the left-skip and right-prefix schedule of Section 4.3 to these data, ignoring early YES answers and the operation cap. Then its number T of parent-generation tasks, its total parent-generation work W_{parent} , and the total input-array length N_{check} of its final checks satisfy

$$T \leq 2q + 2(m+1)U_R/C, \tag{19}$$

$$W_{\text{parent}} \leq H(2q + 2(m+1)U_R/C) + K(J_L + (m+1)J_R), \tag{20}$$

$$N_{\text{check}} \leq U_L + (2m+3)U_R. \tag{21}$$

Every reuse of a left array counts separately in N_{check} .

Proof. Task starts. At a fixed prefix depth, different right tasks have disjoint sets of possible entries: different buckets are disjoint, and different prefixes of the same length are disjoint within a bucket. Each overflow witnesses more than C entries of $\mathcal{U}_R$. There are consequently at most U_R/C overflow nodes at that depth, and at most $(m+1)U_R/C$ over all depths. There are at most q left tasks and q right root tasks. Every other task is one of at most two children of an overflow node. This proves (19), including the possibility that an overflowing full-mask node is dropped without children.

Raw occurrences. Each left occurrence has one bucket and can appear in at most one left task. A right occurrence has one bucket and one prefix at each depth, so it is a potential occurrence in at most $m+1$ right tasks. This counts occurrence identities, rather than their possibly equal weights or masks. It also counts invalid occurrences: their top masks are defined before the local overlap test, and incompatible prefix choices are excluded before the join. Thus the sum of all potential occurrence counts is at most $J_L + (m+1)J_R$. Summing the assumed stopped-task costs gives (20). Only the overhead H is multiplied by the number of tasks.

Final-check inputs. Completed right arrays contain disjoint sets of entries. Different buckets are disjoint. Within a bucket, completed prefixes are incomparable, since a completed task has no descendants, so their mask classes are disjoint as well. Right arrays therefore contribute at most U_R to the total check-input length.

Left arrays used at right-root checks have total length at most U_L , since each bucket supplies at most one such check. Every other check uses a left array of length at most C , and there are at most $2(m+1)U_R/C$ nonroot tasks. Their repeated left lengths total at most $2(m+1)U_R$. Adding these three contributions gives (21). $\square$

Applying the accounting bounds. Every raw four-leaf tuple has a well-defined top mask because $T_1, \dots, T_4$ partition M_0 and each occurs in exactly one leaf on each side. The union is defined even if local-mixer masks overlap. Testing a prefix in its responsible leaves therefore gives exactly the restricted occurrence collection required by Lemma 4.2. The dictionary removes duplicate keys only after the local disjointness test. This verifies the combinatorial premises of the lemma for the concrete schedule.

The stopped-task cost. For sufficiently large main-branch inputs, its cost premise holds with

$$H = u^8(2^{2n} + G), \quad K = u^8. \quad (22)$$

Here is the implementation behind those constants. The four raw leaf arrays in a task have total length $A < 4 \cdot 2^{185n}$ by (15); regenerating them, including their masks and prefix tests, takes $O(n^2A)$ operations. Sorting and stream initialization cost $O(nA)$, and the two child streams together have at most G occurrences. Their arrays and mutable states occupy $O(A)$ words, which is at most B for large n . The tie buffer has length B , and Lemma 2.4 charges all replay, including stopped replay, to stream advances and potential matching pairs. Dictionary initialization can be bounded by $O(u(C+1))$, and local mask checks and trie operations cost $O(u)$ per reported tuple. For Z potential matching tuples, all this gives

$$O(n^2A + B + nG + u(C+1) + uZ).$$

Since $C \leq 2^{199n}$, $B \leq 2^{19n}$, and $\log u \leq n/10^9$, this is at most $H + KZ$ with (22), after enlarging the absolute main-branch threshold. The raw term includes tuples later rejected for overlap and repeated occurrences. A small value of U_j does not bound this term.

Thus Lemma 4.2 controls the complete schedule before any probability estimate is used. The numerical bounds for G, J_j, U_j and the proof that a solution reaches a completed right prefix belong to Section 6.2. The remaining independent ingredient is the final check on two bounded arrays, which we now construct.

5 Weighted disjointness in bounded space

The parent schedule produces two arrays of records, each consisting of an actual weight and a top-mixer mask. We want one record from each array whose weights sum to a given target and whose masks are disjoint. A separating set turns this into an ordinary weight search: retain the left records whose sets lie inside it, and the right records whose sets lie outside it. Every pair of retained records is then disjoint. Sorting their weights finds an exact complementary pair, if one exists.

The simplest illustration uses a block of g indices, with g divisible by four. Suppose each side of a fixed disjoint pair occupies $g/4$ indices. A uniformly sampled set of size $g/2$ separates this pair

with probability

$$\frac{\binom{g/2}{g/4}}{\binom{g}{g/2}}.$$

Indeed, the sampled set must include the left $g/4$ indices, exclude the right $g/4$ indices, and choose its remaining $g/4$ elements from the $g/2$ unused indices. Sampling a family whose size is a constant times the reciprocal of this probability gives a constant chance of covering the fixed pair. An individual record can occur in several members of this family; those occurrences determine the work.

We will apply this idea on four blocks. The left and right cardinalities may differ, so the sampled set size is biased to control occurrences on both sides. We first specify the distribution. We then show how to search its useful index tuples without storing all tuples or testing all pairs of records.

This construction follows the separating-set approach of Nederlof and Węgrzycki [9, full version, Section 6], which builds on the separating collections of Fomin, Lokshtanov, Panolan, and Saurabh [6, Section 4.2, Definition 4.3 and Lemma 4.5]. We use the affine bias from Nederlof and Węgrzycki's construction [9, full version, Lemma 6.6], with the cardinality-range extension of its entropy estimate by Belova, Chukhin, Kulikov, and Mihajlin [3, full version, Section 4.7]. The rounding, traversal, and exact estimate required here are proved below.

5.1 Inputs and sampled families

Recall the parameters u, m, g, C from (1). The four blocks in the representation layout give the partition

$$M_0 = T_1 \sqcup T_2 \sqcup T_3 \sqcup T_4, \quad |T_i| = g.$$

Write $k = k_0/m \in [1/6, 1/4]$, where k_0 is the prescribed left mask size. To recall the balanced block counts explicitly, write $k_0 = 4d + r$ with $0 \leq r < 4$ and put $c_i = d + 1$ for $i \leq r$ and $c_i = d$ otherwise. Then $\sum_i c_i = k_0$ and $|c_i - kg| < 1$. The left array $\mathcal{L}$ and right array $\mathcal{R}$ are nonempty and have lengths at most C . Their records are (x, D) and (y, E) , respectively, with $D, E \subseteq M_0$ and

$$|D \cap T_i| = c_i, \quad |E \cap T_i| = g/2 - c_i \quad (1 \leq i \leq 4). \quad (23)$$

These conditions do not assert that an arbitrary pair is disjoint. For the transformed target t' , the required output is YES only if some pair satisfies

$$D \cap E = \emptyset, \quad x + y = t'. \quad (24)$$

Array lengths count occurrences; distinct records are not required.

The actual weights and target have magnitude below $2^{b + \lceil \log(n+2) \rceil + 4}$. The exact arithmetic, masks, and array indices used here fit in a constant number of the words specified in Section 1; all stored tables and retained random choices count as workspace. The space bound for this check will hold on every outcome of the randomness.

Let

$$\alpha = \log 3, \quad \alpha_m = \frac{\lfloor \log(3^m) \rfloor}{m}, \quad e = \left\lfloor g \left(\frac{1}{2} + \alpha_m \left(k - \frac{1}{4} \right) \right) \right\rfloor.$$

The logarithm floor in α_m is the bit length of the integer 3^m minus one; no real-number oracle is needed. For each block set

$$\nu_i = \left\lceil 10 \binom{g}{e} / \binom{g/2}{e - c_i} \right\rceil, \quad \mathcal{F}_i = (S_{i,1}, \dots, S_{i,\nu_i}). \quad (25)$$

All $S_{i,j}$ are independent, and $S_{i,j}$ is a uniform e -element subset of T_i . The families are indexed: equal sampled sets retain their separate indices. The arrays and target are fixed independently of all four families.

The binomial arguments in (25) are feasible for large n . To see this, put

$$\zeta = \frac{1}{2} + \alpha \left(k - \frac{1}{4} \right), \quad Y = \zeta - k = \frac{1}{4} + (\alpha - 1) \left(k - \frac{1}{4} \right).$$

Then $e = \zeta g + O(1)$ and $c_i = kg + O(1)$, uniformly in $k \in [1/6, 1/4]$. Since $1 < \alpha < 2$, Y stays strictly between 0 and $1/2$ throughout this interval.

As in Lemma 2.2, h denotes binary entropy and $h_* = h(1/4)$.

Lemma 5.1 (Weighted disjointness). *For the input and independent families just specified, there is a weighted-disjointness check with the following guarantees, for all sufficiently large n .*

1. *It returns YES only for a pair satisfying (24). For every fixed pair satisfying that relation, the probability that the check finds a solution is at least $1 - 4e^{-10}$.*
2. *Preprocessing the families takes at most $\text{poly}(n)2^{2g}$ operations and words. Thereafter the expected check work is at most*

$$(|\mathcal{L}| + |\mathcal{R}|) u^{22} 2^{(1-h_*)m}.$$

3. *Workspace, including preprocessing, is deterministically at most $O(nC + n^2 + 2^{2g})$ words. Any execution stopped early has no larger work or space usage than the complete traversal used in these bounds.*

The probability statements concern the stated ideal uniform families. Generating them with at most n^3 attempts per ordinary rejection draw also has deterministic cost $\text{poly}(n)2^{2g}$, or reports sampling failure. It can be coupled to the ideal families with failure probability $\exp(-\Omega(n^2))$.

Proof. We first check coverage of one fixed pair. We then describe a traversal whose workspace is bounded for every family outcome. Its time will be charged to individual occurrences of input records, and an entropy estimate will bound their expectation.

Coverage of one pair. Fix a pair satisfying (24). On block T_i , a sampled set must contain its left mask and avoid its right mask. Their union has size $g/2$, so the separation probability is exactly

$$\theta_i = \binom{g/2}{e - c_i} / \binom{g}{e}.$$

Equation (25) gives $\nu_i \theta_i \geq 10$. The probability that no family member separates the pair on this block is at most $(1 - \theta_i)^{\nu_i} \leq e^{-10}$. A union bound over the four blocks shows that some tuple of family indices separates the whole pair with probability at least $1 - 4e^{-10}$. This is coverage of a fixed pair; no simultaneous coverage assertion for every pair is needed.

Preprocessing and the traversal. A left record is compatible with index j on block i when $D \cap T_i \subseteq S_{i,j}$; a right record is compatible when $(E \cap T_i) \cap S_{i,j} = \emptyset$. For every mask on each block, store both sorted lists of compatible indices, one for containment and one for avoidance. Since $\binom{g/2}{e - c_i} \geq 1$,

$$\nu_i \leq 10 \cdot 2^g + 1.$$

Store a block mask and a family index in one word each. For each block and each of its 2^g masks, the two incidence lists together need at most $2\nu_i$ index words; their offsets and lengths use only a constant number of words. Across the four blocks, a flat array of capacity at most $8 \cdot 2^g(10 \cdot 2^g + 1)$ suffices for their contents. Generating the families needs $O(2^g + n)$ words, including the current permutation. Local-index maps and other control data use at most $O(n^2)$ words. Thus the families and incidence tables occupy $O(2^{2g} + n^2)$ words. Filling the lists in increasing index order costs $\text{poly}(n)2^{2g}$ operations. A binary search in the stored list decides whether a record has a compatible index in any given interval of family indices.

Process the blocks in order. For the current block, begin with its whole index interval $[1, \nu_i]$. Filter both received arrays to records having a compatible index in this interval. If either filtered array is empty, finish this branch. Otherwise split a nonsingleton interval into two nearly equal intervals and visit both children. At a singleton, fix that family index and enter the next block with the filtered arrays. After fixing all four indices, sort the two arrays by actual weight and search for weights summing exactly to t' .

Every pair at this last search is disjoint, because its left mask lies inside the chosen set and its right mask lies outside it on every block. Thus a reported YES is valid. A fixed pair separated by a tuple of indices remains in both arrays on the entire path to that tuple. It therefore reaches an exact-weight search, unless a previous search has already returned YES. This proves the coverage guarantee.

Use depth-first traversal and retain the parent arrays until their children finish. Each depth has its own buffers, written only up to their current lengths. In particular, a small filtered array does not require clearing a length- C buffer. The combined depth of the four interval trees is at most

$$4 + \sum_{i=1}^4 \lceil \log \nu_i \rceil \leq m + 20 = O(n).$$

At each depth at most two arrays of length C are retained, and each record occupies a constant number of words. The retained-buffer term is therefore $O(nC)$ words, including the input arrays. Sorting at a terminal branch uses only $O(C)$ additional words, and stack control uses $O(n)$ words. Together with the incidence tables this gives the explicit bound

$$O(nC + n^2 + 2^{2g}) \quad \text{writable words.} \tag{26}$$

This bound applies to every choice of the families and every feasible input array. Any other arrays retained by the caller are additional.

Charging the visited occurrences. Fix one input occurrence, and let t_i be its number of compatible indices in family i . For a compatible prefix of previously fixed indices, the occurrence is scanned at the next block's root. If $t_i = 0$, that scan removes it. If $t_i > 0$, it can pass the filter only on paths to its t_i compatible leaves. The union of these paths has $O(t_i(1 + \log \nu_i))$ nodes; scans into their children have the same order of cost. There are at most $\prod_{r < i} t_r$ compatible prefixes before block i . Its scans at that block are consequently bounded by a polynomial factor times

$$\left(\prod_{r < i} t_r \right) (1 + t_i).$$

The term 1 is necessary even when $t_i = 0$.

An interval predicate costs $O(n)$ operations, including binary search and local-mask extraction, and each tree has depth $O(n)$. After the fourth block, sorting and searching cost $O(n)$ operations

per surviving occurrence. These costs give an $O(n^2)$ factor in the preceding scan count. Node overhead can be charged to scans because a child or new block is entered only with nonempty arrays. For sufficiently large n , the complete charge to this input occurrence is at most

$$u^4 \sum_{j=0}^4 \prod_{i=1}^j t_i, \quad (27)$$

where the empty product equals one. This charge bounds a traversal which completes all branches even after finding a solution. Stopping at a YES, an operation limit, or any earlier point can only reduce its work.

Expected incidences on one block. For a fixed left mask of size $c = c_i$, the probability of containment in a uniform e -set is $\binom{g-c}{e-c} / \binom{g}{e}$. Since $\nu_i \leq 11/\theta_i$,

$$\mathbb{E} t_i \leq 11 \binom{g-c}{e-c} / \binom{g/2}{e-c}. \quad (28)$$

Apply the binomial bounds of Lemma 2.2. Set

$$X = 1 - k, \quad Y = \frac{1}{4} + (\alpha - 1) \left(k - \frac{1}{4} \right), \quad E(k) = X h(Y/X) - \frac{1}{2} h(2Y).$$

Those bounds show that the logarithm of the right-hand side of (28) is at most

$$gE(k) + O(\log n). \quad (29)$$

The bound is uniform in $k \in [1/6, 1/3]$. Indeed, Y lies between k and $1/4$, and Y/X and $2Y$ stay away from zero and one on this compact interval. Their entropy derivatives are bounded in a fixed neighborhood. The $O(1)$ rounding of c and e therefore changes each binomial entropy exponent by only $O(1)$. The lower binomial bound for the denominator contributes at most $\log(g/2 + 1)$ in addition to constants.

For right records use the complements of the sampled sets. Write

$$k' = \frac{1}{2} - k, \quad c' = g/2 - c, \quad e' = g - e.$$

The complemented sets are uniform e' -sets. Their ideal bias is $1 - \zeta = 1/2 + \alpha(k' - 1/4)$, and rounding again costs only a bounded amount. The family-size denominator stays the same because

$$e' - c' = g/2 - (e - c), \quad \binom{g/2}{e' - c'} = \binom{g/2}{e - c}.$$

Thus (28)–(29) apply to right records with $k' \in [1/4, 1/3]$. It remains to prove one sharp entropy bound covering both sides.

The exact entropy bound. We claim

$$E(k) \leq 1 - h_* \quad (1/6 \leq k \leq 1/3). \quad (30)$$

Let $F = (\ln 2)E$, $Z = X - Y$, and $W = 1/2 - Y$. Expanding entropy gives

$$F = X \ln X - Z \ln Z - \frac{1}{2} \ln \frac{1}{2} + W \ln W.$$

Consequently

$$F' = \ln \frac{Z}{X} + (\alpha - 1) \ln \frac{Z}{W}, \quad F'' = \frac{1}{X} - \frac{\alpha^2}{Z} + \frac{(\alpha - 1)^2}{W}.$$

All denominators are positive even on $[0, 1/2]$: explicitly,

$$Z = \frac{2 + \alpha}{4} - \alpha k, \quad W = \frac{\alpha}{4} - (\alpha - 1)k, \quad 1 < \alpha < 2.$$

The ratio Z/W decreases on this interval, since

$$\left(\frac{Z}{W} \right)' = \frac{\alpha - 2}{4W^2} < 0.$$

Hence $Z/W \leq (2 + \alpha)/\alpha$. Also $Z/X \leq 1$, since $Y > 0$. Multiplying the second derivative by Z now gives

$$ZF'' = \frac{Z}{X} - \alpha^2 + (\alpha - 1)^2 \frac{Z}{W} \leq 1 - \alpha^2 + (\alpha - 1)^2 \frac{2 + \alpha}{\alpha} = -2 + \frac{2}{\alpha} < 0.$$

Thus E is strictly concave. At $k = 1/4$ we have $X = 3/4$, $Z = 1/2$, and $W = 1/4$, so $F' = \ln(2/3) + (\alpha - 1) \ln 2 = 0$. Its maximum is therefore

$$E(1/4) = \frac{3}{4}h(1/3) - \frac{1}{2} = 1 - h(1/4).$$

The last identity follows from $h(1/3) = \log 3 - 2/3$ and $h(1/4) = 2 - (3/4) \log 3$. This proves (30) without a positive error in its exponential rate.

Combining (28)–(30), for either side and every block,

$$\mathbb{E}t_i \leq u^4 2^{(1-h_*)g}$$

for sufficiently large n . The four block families are independent, so expectations of the products in (27) factor. Since $1 - h_* > 0$ and $m = 4g$, each of the five terms in (27) has expectation at most $u^{20} 2^{(1-h_*)m}$. Their sum is at most $u^{22} 2^{(1-h_*)m}$ for large n . Summing over input occurrences proves the expected work bound. The preceding construction already proved the deterministic space bound.

Finally, a uniform e -set is obtained by taking the first e positions of a uniform permutation of its block. Generate each permutation from fresh successive range draws, with the bounded implementation of Lemma 7.1. Across all four families there are at most

$$4g(10 \cdot 2^g + 1)$$

such calls. Its conditional per-call bound gives a coupling failure probability at most $4g(10 \cdot 2^g + 1)2^{-n^3}$, which is $\exp(-\Omega(n^2))$. The bound uses the ideal product law and subtracts the coupling failure; it does not condition that law on successful bounded sampling. Allowing all n^3 attempts per draw adds only a polynomial factor to the generation work, which fits $\text{poly}(n)2^{2g}$ together with preprocessing. $\square$

5.2 Reusing the same families

The check can be called repeatedly with the same four families. Different calls are then correlated, but their work still adds by linearity of expectation. The necessary condition is that the full list of their input arrays be fixed independently of those families.

Corollary 5.2 (Reusing the families). *Fix a finite list of checks satisfying the input conditions of Lemma 5.1, with common block parameters, and let*

$$N_{\text{check}} = \sum_{\text{checks}} (|\mathcal{L}| + |\mathcal{R}|).$$

The list may itself be random, provided that, conditional on the list and its parameters, the four families have the independent distributions (25). In the ideal experiment, the expected total check work conditional on the list and its parameters is at most

$$W_0 = N_{\text{check}} u^{22} 2^{(1-h_*)m}.$$

Preprocessing is paid once. Processing checks one at a time uses the check workspace of Lemma 5.1, in addition to any storage retained by the caller. For any $0 < \delta < 1$, total check work exceeds W_0/δ with probability at most δ . Suppose a designated check and a solution pair in it are also chosen, and that conditional on the list, its parameters, and these choices, the families still have the independent distributions (25). Then with probability at least $1 - 4e^{-10} - \delta$ the pair is covered and the total check work is at most W_0/δ .

Proof. An empty list has $N_{\text{check}} = 0$ and zero check work. Otherwise condition on the complete list and its parameters, and apply Lemma 5.1 to each input occurrence and sum expectations. Markov's inequality gives the work bound. For the last assertion, condition additionally on the designated check and pair. The family law is unchanged, so the same expected-work bound holds and the pair has coverage probability at least $1 - 4e^{-10}$. A union bound intersects the coverage and work events; those events need not be independent. $\square$

In the Subset Sum application, continue parent generation as if checks never returned YES and as if no operation cap were present, retaining all dictionary overflow rules. This defines the complete check list without consulting the separating families. Conditional on the layout, their sampled product law is independent of all parent-generation choices. It therefore remains unchanged after fixing that list and a designated check and witness selected from it using only those choices. Corollary 5.2 applies once Section 6.2 bounds the total input length of this list. The actual run visits only an initial portion of its work; the list itself is never stored, and coverage is required only for the selected pair.

6 Survival within the resource budgets

The preceding construction always respects its dictionary capacities, but it may discard a bucket containing a solution or stop at the operation cap. We now show that a fixed solution reaches a completed check within the cap with inverse-polynomial probability, enough for repetition to succeed. The order of the argument matters: first fix a surviving representation, subtract an unconditional resource-failure bound, and only then use the extra prime and separating families.

6.1 Fixing a witness before the last random choices

Throughout this section suppose that the input is a YES instance and $\mu < .00765n$, where μ is the mean deficiency of a uniform size- s subset of the input indices. We first use ideal exact sampling. The extra prime q and the separating families are not exposed in this subsection. Our objective is to select one representation before either of those sources of randomness is used. We first balance the fixed solution inside the mixers, retain many distinct choices with the prescribed leaf counts, and

finally show that the low residues hit some of them. Every selection of a witness in this argument is an analysis choice; the program enumerates candidates without knowing it.

Lemma 6.1. *For a main-branch YES input with $\mu < .00765n$, ideal sampling produces, with probability at least u^{-100} , a passing spread test and a representation of a solution satisfying all prescribed leaf counts and all low modular filters. Its two parent weights x_L, x_R , with $x_L + x_R = t'$, and all of its masks can be fixed as functions of the nonfamily choices preceding q . Conditional on these choices, q still has its prescribed independent prime law; the separating families also retain their prescribed product law, with the now fixed layout parameters.*

Proof. Balance the solution and its deficiencies. Fix an original solution S for analysis. Under the random sign flip F , its signed counterpart $S' = S \triangle F$ is a uniform subset of the input indices, independent of the mixers. Put $A_i = S' \cap M_i$. By Lemma 2.2, the balance event

$$|A_0| = |A_L| = |A_R| = s$$

has probability at least $(m+1)^{-3}$. This event has the same probability for every choice of the three mixers before ordering them. Conditional on it, their distribution is unchanged, and their selected halves are independent uniform halves given the mixers. Each selected half and its complement in a pre-order mixer is therefore marginally a uniform size- s set of input indices.

By Lemma 2.1, with conditional probability $1 - O(1/n)$, all six halves have deficiency at most $\tau = .0078n$: a deviation of $.0001n$ above their common mean already suffices. The same lemma, now for size- m sets, shows that each pre-order mixer's deficiency is within $.0002n$ of the size- m mean, again with conditional failure probability $O(1/n)$. Thus the spread test (11) passes jointly with these events for sufficiently large n . These are statements about the original weights, equally valid after sign changes by deficiency invariance. For this analysis we may sample the signs even on choices on which the algorithm would have stopped at the spread test.

If A, B partition a mixer M into halves, then $N(M) \leq N(A)N(B)$, so

$$d(A) + d(B) \leq d(M).$$

At least one half has deficiency at most $d(M)/2$. Given the three unordered pairs of halves, each orientation as the selected half is uniform, independently across the mixers. Both preceding good events concern the original-weight deficiencies of the unordered halves and are invariant under these orientations, as is the ordering of the mixers. With further conditional probability at least $1/8$, then,

$$d(A_i) \leq f_i = \min(\tau, d_i/2) \quad (i = 0, L, R). \quad (31)$$

No conditional independence of the signs is asserted or needed here. Sign invariance and (31) give at least $2^{s-f_i} = 2^{L_i}$ distinct signed subset sums on A_i .

Retain representatives with the prescribed leaf counts. Partition these sums according to subset size. Some size has at least $2^{L_i}/(s+1)$ distinct sums. Choose a maximizing size at most $s/2 = m/4$, using complementation in A_i , which preserves the number of distinct sums between complementary sizes. This size exceeds $m/6$ for sufficiently large n . Otherwise its number of subsets would be at most $2^{sh(1/3)}$, whereas

$$\frac{s - \tau - \log(s+1)}{s} \longrightarrow .935 > \frac{14}{15} > h(1/3).$$

For each mixer fix such a size and one subset for each distinct sum of that size. These are analysis choices made before the size guesses and subblock partitions. The probability that all three guesses k_i equal these sizes is at least $(m+1)^{-3}$.

We next retain many of these representatives with the required finer counts. Fix one representative $D \subset A_i$, and write $l = 4$ for the top mixer and $l = 2$ for either other mixer. In a uniform partition into l labeled equal subblocks, prescribe the counts of the three categories

$$D, \quad A_i \setminus D, \quad M_i \setminus A_i$$

to be respectively the smaller-mask counts, their prescribed complementary counts, and an equal split of the outside category. These vectors fill every block exactly. In a top block their counts are $c_a, g/2 - c_a, g/2$; in a local block they are $a_{jr}, s/2 - a_{jr}, s/2$. The divisibility of m by eight makes all these complementary counts integral. Each category's counts are as equal as possible. For v objects, a most balanced multinomial coefficient is maximal, since transferring one object from a larger count to a count smaller by at least two increases the coefficient. There are at most $(v+1)^l$ possible count vectors, and their coefficients sum to l^v . Each of our three favorable coefficients is therefore at least $l^v/(v+1)^l$. Dividing their product by the multinomial coefficient for the whole partition, which is at most l^m , shows that D survives with probability at least

$$(m+1)^{-3l} \geq u^{-3l}.$$

This survival ensures the finer counts for both D and its complement in A_i .

Let $Z \in [0, 1]$ be the surviving fraction of a representative family. If $\mathbb{E}Z \geq p$, then $\mathbb{E}Z \leq p/2 + \Pr[Z \geq p/2]$; hence $\Pr[Z \geq p/2] \geq p/2$. Apply this with $p = u^{-3l}$. The three subblock partitions are independent, so with further probability at least $u^{-24}/8 \geq u^{-25}$, all three retain these fractions. Let X_i consist of the distinct signed weights of the surviving smaller masks, retaining their associated masks. For sufficiently large n ,

$$|X_i| \geq \frac{2^{L_i}}{u^{14}}. \quad (32)$$

We have now retained many distinct weights whose masks already meet all cardinality prescriptions. It remains to ensure that the low modular filters keep compatible choices. The next argument first bounds collisions under the random moduli, and then exposes the uniform target residues in the order that fixes the top choice before its children's choices.

Hit occupied low-residue classes. This collision-to-occupancy argument is the residue-coverage method of Austrin, Kaski, Koivisto, and Nederlof [2, Proposition 3.5] and its product-modulus version in Nederlof and Węgrzycki [9, full version, Lemma 3.2]; we give the bounds for the present nested moduli explicitly. Expose the prime factors forming p_i . For fixed X_i , the sum of squares of its residue-class sizes modulo p_i has expectation at most

$$|X_i| + |X_i|^2 u^{10} 2^{-L_i} \leq u^{15} |X_i|^2 2^{-L_i},$$

by (17) and (32). Markov's inequality and a union bound show that, with probability at least $1 - 3/u$, all three sums of squares are at most $u^{16} |X_i|^2 2^{-L_i}$. The moduli need not be independent of one another. Cauchy-Schwarz gives at least

$$\frac{2^{L_i}}{u^{16}} \geq \frac{p_i}{u^{16}} \quad (33)$$

occupied residues for each X_i .

First expose ρ_0 . Assign a surviving smaller mask $D_0 \subset A_0$ to L and its complement in A_0 to R. As D_0 varies, the left solution-parent weight is its weight plus a fixed term: the selected left background weight and the weight of A_L . Equation (33) implies that a uniform $\rho_0 \bmod p_0$ hits

one of these weights with probability at least u^{-16} . On a hit, fix a representative deterministically, thereby fixing disjoint top masks and parent weights $x_L, x_R = t' - x_L$.

Splitting A_j between the two children cannot change parent j 's weight: their union still uses all of A_j . For each side j , assign a surviving smaller mask $D_j \subset A_j$ to its first child and the complement to its second child. The first-child weight is the weight of D_j plus a now fixed term from its background and its portion of the top mask. Expose the independent residues ρ_L, ρ_R . They both hit possible first-child residues with probability at least u^{-32} . The second-child filters then follow automatically from $p_j \mid p_0$ and the parent filters. All prescribed leaf counts hold by the surviving subblock choices; the selected solution has no background cardinality constraint to satisfy.

The product of the conditional probability lower bounds, for large n , is at least

$$u^{-3} \cdot \frac{1}{2} \cdot \frac{1}{8} \cdot u^{-3} \cdot u^{-25} \cdot \frac{1}{2} \cdot u^{-48} \geq u^{-100}.$$

All choices of representatives on hits may be deterministic analysis choices. They precede both q and the separating families. For any subsequent q , one loop iteration has $v_L = x_L \bmod P$ and $v_R = x_R \bmod P$. $\square$

6.2 Unconditional resource bounds

A surviving representation must reach a completed check before the operation cap. Recall the three ambient counts from the prefix analysis: G counts low-filtered raw child pairs, J_j counts low-filtered raw four-leaf tuples on side j , and U_j counts distinct valid weight-top-mask entries passing only the parent low filter. None uses q , a prefix restriction, or a separating family. The first two retain all occurrence multiplicities, including tuples with a local overlap. The last counts only valid entries and ignores the child filters. We shall bound these counts without conditioning on the rare survival event.

For their first moments, condition on a passing mixer setup, signs, guesses, layout and allowed moduli p_i , leaving the low residues ρ_0, ρ_L, ρ_R independent and uniform. Each child's required residue is marginally uniform modulo p_j , including child 2 because ρ_j is independent of ρ_0 . Each fixed four-leaf tuple passes its parent filter and first-child filter with probability $1/(p_0 p_j)$; divisibility $p_j \mid p_0$ then gives the second-child filter. Before its parent filter, the number of distinct valid entries on side j is at most

$$2^{r_j} 2^{H_{0j}m} 2^{m-d_j}.$$

Here the three factors count background choices, possible top masks, and distinct actual weights from disjoint use of M_j . The last factor is N_j , also for the signed weights. Each entry passes the parent filter with probability $1/p_0$.

Define the analysis scales

$$\begin{aligned} \Gamma &= \frac{n}{4} + \left(-\frac{5}{4} + \frac{3}{2}h_*\right)m + f_{\max}, \\ d_* &= \min(d_L, d_R), \\ \Lambda &= \frac{n}{2} + (\overline{H}_0 - 1)m - d_* + f_0. \end{aligned} \tag{34}$$

By (13), (14), and (17), the conditional moments satisfy

$$\mathbb{E}G \leq u^{26}2^\Gamma, \quad \mathbb{E}(J_L + J_R) \leq u^{51}2^{n/2}, \quad \mathbb{E}U_j \leq u^{10}2^\Lambda. \tag{35}$$

For the first bound, use $\beta_j - L_j \leq \Gamma$, the u^{20} factor for raw pairs, and the u^5 loss in the modulus lower bound. For the second, raw four-leaf tuple counts are at most $u^{40}2^{2\beta_j}$, and their exponent after filtering obeys

$$\begin{aligned} 2\beta_j - L_0 - L_j &\leq \frac{n}{2} + \left(-\frac{5}{2} + 3h_*\right)m + 2\tau \\ &\leq \frac{n}{2}. \end{aligned} \tag{36}$$

The final inequality holds for large n , since the excess is at most $-.0658m + .0156n$, and $m = .24n + O(1)$. For the third bound, the exponential part of the preceding universe count divided by the parent modulus is

$$\frac{r}{2} + \overline{H}_0 m + m - d_j - L_0 = \frac{n}{2} + (\overline{H}_0 - 1)m - d_j + f_0 \leq \Lambda;$$

the allocation's logarithmic error and the modulus loss fit in the factor u^{10} .

Let $\mathcal{E}$ be the resource event

$$G \leq u^{200}2^\Gamma, \quad J_L + J_R \leq u^{200}2^{n/2}, \quad U_L, U_R \leq u^{200}2^\Lambda. \tag{37}$$

The scales in (34) are fixed under the conditioning used in (35). Markov's inequality and a union bound, followed by averaging over that conditioning, give

$$\Pr[\text{spread test passes and } \mathcal{E}^c] \leq u^{-174} + u^{-149} + 2u^{-190} < \frac{1}{10}u^{-100}. \tag{38}$$

This is a joint failure estimate, not an estimate conditioned on the rare event in Lemma 6.1. Consequently that lemma's surviving-representation event and $\mathcal{E}$ hold together with probability at least $.9u^{-100}$ in the low-mean YES case.

Three exponent relations control the subsequent costs:

$$\xi + \Gamma \leq \frac{n}{2}, \quad \Lambda + (1 - h_*)m \leq \frac{n}{2}, \quad \xi - (\Lambda - .199n) \geq .0015n. \tag{39}$$

The first relation pays for scanning the low-filtered children over roughly 2^ξ tasks. The second pays for the repeated separator incidences of distinct records. The third makes the expected number of different-weight records in a solution bucket smaller than its capacity C . These relations hold for large n on every passing setup. The first follows from $.032 - 1.25 + 1.5h_* < 0$, with cancellation of $f_{\max}$. For the second, use $\overline{H}_0 \leq h_*$ and $f_0 \leq d_*$, since M_0 has least deficiency. For the third, direct subtraction gives the lower bound

$$-.051n + (1.032 - h_*)m + d_* - f_0 - f_{\max}.$$

Here $f_0 \leq d_*/2$, while the spread test implies $f_{\max} \leq d_*/2 + .00025n$. Since $1.032 - h_* > .2206$ and $m = .24n + O(1)$, this lower bound is at least $.0015n$. The first two relations preserve the exact time exponent $1/2$; no positive exponential slack is added there.

6.3 The extra prime and the solution bucket

Condition on a surviving representation and $\mathcal{E}$, exposing all choices so far but still not q . In its loop iteration, the expected number of entries in side j 's universe with weight different from x_j but congruent to it modulo $P = p_0q$ is at most

$$U_j u^{32-\xi} \leq u^{203}2^{.199n-.0015n}. \tag{40}$$

The nonzero difference must be divisible by the independent prime q , so the prime collision bound applies. No coprimality of q and p_0 is required. Since $C \geq 2^{.199n-1}$, the right side of (40) divided by $C/2$ is at most

$$2^{(203/10^9 - .0015)n+2}$$

on the main branch. Markov's inequality shows, with conditional probability at least .9 for large n , that both sides have at most $C/2$ such different-weight entries.

Equal-weight entries need a separate argument. On L, at most $\binom{m}{k_0} \leq 2^{h_* m} \leq 2^{.194736n} < C/2$ distinct entries can have weight x_L , since the smaller top mask has size at most $m/4$. Thus the entire solution L bucket fits. On R, follow the prefix of the solution top mask. At a full prefix there is at most one entry of weight x_R , by deduplication. Together with the different-weight bound, this full-prefix bucket fits, so the solution's prefix path terminates in a completed nonoverflow task. In every task compatible with the solution masks, its surviving leaf choices generate the corresponding parent entries. Hence the solution pair reaches a weighted disjointness check, unless an earlier YES or an operation or sampling abort has already occurred.

6.4 Applying the three prefix bounds

Consider the complete hypothetical parent schedule obtained by ignoring YES answers and the operation cap, while retaining the actual overflow rules. Its generation uses only the parent choices and q . Thus it can be fixed without examining any separating family. The three bounds in Lemma 4.2 now have their required numerical inputs; we apply each to the quantity it controls.

First, the number of parent-generation tasks is at most

$$T \leq 2q + 2(m+1)U_R/C \leq u^{203}2^\xi. \quad (41)$$

Indeed, $q \leq 2^\xi$, $C \geq 2^{.199n-1}$, and the last inequality in (39) gives $U_R/C \leq 2u^{200}2^{\xi-.0015n}$ on $\mathcal{E}$. The main-branch guard absorbs the remaining polynomial factor.

Second, the concrete stopped-stream implementation supplies

$$H = u^8(2^{.2n} + G), \quad K = u^8 \quad (42)$$

in the task-cost formula of Lemma 4.2. The term H pays for regenerating the leaves, prefix tests before streaming, sorting, stream initialization and traversal, the bounded tie buffer, and dictionary initialization. The term K pays for each potential matching tuple, including replay, invalid local overlaps and repeated entries. Consequently the full parent-generation work is at most

$$\begin{aligned} HT + K(J_L + (m+1)J_R) &\leq u^{211}2^\xi(2^{.2n} + u^{200}2^\Gamma) + u^8(J_L + (m+1)J_R) \\ &\leq u^{420}2^{n/2} \end{aligned}$$

for sufficiently large main-branch inputs. Here $\xi + .2n < n/2$ follows from $\xi \leq .25768n$, whereas $\xi + \Gamma \leq n/2$ is the first exact relation in (39). The raw-match term is charged by prefix depth; it is never multiplied by T .

Third, write N_{check} for the total array length supplied to all final checks, counting every reuse of a left array. The final bound of Lemma 4.2 gives

$$N_{\text{check}} \leq U_L + (2m+3)U_R. \quad (43)$$

Completed right chunks pay for their own entries, and overflow entries pay for repeated left arrays. This is why the final checks use the distinct-entry counts U_j , even though generation required the raw counts J_j .

6.5 Independent families and bounded-trial success

Fix all choices through q , on the joint survival, resource and bucket events. Choose deterministically the check on the completed solution-prefix path and the solution pair in it. The complete hypothetical check list, its layout parameters, this designated check and its pair are all functions of those nonfamily choices. Conditional on the layout, the program sampled the four indexed families from the product law specified in Section 5, independently of every parent and filter choice. Therefore that same product law holds after conditioning jointly on this entire list, its parameters, and the designated pair. This fact, rather than delayed exposure by itself, verifies the hypothesis of Corollary 5.2.

On $\mathcal{E}$, (43) and that corollary give conditional expected total check work at most

$$N_{\text{check}} u^{22} 2^{(1-h_*)m} \leq u^{230} 2^{\Lambda+(1-h_*)m} \leq u^{230} 2^{n/2}.$$

The second exact relation in (39) is used here. Markov's inequality bounds the chance of exceeding $u^{240} 2^{n/2}$ operations by u^{-10} . The designated pair has coverage probability at least $1 - 4e^{-10}$. A union bound gives both coverage and this work bound with conditional probability at least $1 - 4e^{-10} - u^{-10}$; these events need not be independent. The hypothetical list is an analysis device and is never stored.

Lemma 6.2. *On a YES input with $\mu < .00765n$, a representation trial with the bounded samplers and operation cap succeeds with probability at least u^{-110} , for sufficiently large main-branch inputs.*

Proof. Lemma 6.1 and (38) give probability at least $.9u^{-100}$ of simultaneous survival and resources. The independent extra prime gives the bucket event with conditional probability at least $.9$. Given both stages, the family argument gives coverage and check work with conditional probability at least $1 - 4e^{-10} - u^{-10}$. These multiply as conditional lower bounds, not as an assertion that the events are independent. On their intersection, parent generation costs at most $u^{420} 2^{n/2}$ and all checks cost at most $u^{240} 2^{n/2}$.

The remaining setup work is smaller. Counting the three mixer sumsets by half-list streams takes $\text{poly}(n)2^m$ operations. There are constantly many prime calls, each using at most n^3 attempts, with $\text{poly}(n)2^{129n}$ work per attempt. Family sampling and incidence construction take $\text{poly}(n)2^{2g}$ operations, including their bounded range-draw costs. Direct combination generation and weight/mask formation cost at most $O(n^2)$ per item; comparison sorting, heap operations and interval searches cost at most $O(n)$ per item or advance; trie operations cost $O(u)$ per key. These explicit operations fit the displayed loose polynomial powers. Including the worst permitted sampler costs, the joint favorable execution uses fewer than $u^{500} 2^{\lceil n/2 \rceil}$ underlying word operations for large n .

Lemma 7.1 couples the bounded draws to the ideal experiment with loss $\exp(-\Omega(n^2))$ per trial. This is smaller than u^{-110} by any fixed factor on the main branch. Subtracting that loss from

$$.81u^{-100}(1 - 4e^{-10} - u^{-10})$$

leaves at least u^{-110} . On this intersection the trial completes before its cap and finds a solution. An earlier YES is already a success. $\square$

7 A single uniform bounded algorithm

We now implement the ideal choices, bound every stored structure on every execution, and finish the proof of Theorem 1.1. The program runs both trials; the deficiency split selects a success proof, not a branch that the program must recognize. Its sampling limits and operation counters enforce finite running time even when the resource event of Section 6.2 fails.

The full schedule is as follows. On either exceptional case in the guard, enumerate all indexed subsets exactly. Otherwise perform u^{120} rounds. In each round run the compressed-block trial and then the representation trial, sequentially, using fresh randomness and reset trial state. Abort each trial after $u^{500}2^{\lceil n/2 \rceil}$ underlying word operations; a sampling failure has the same effect as an abort. A YES from either trial stops the program. If no trial returns YES, return NO after the last round.

7.1 Bounded sampling and its exact law

An ideal draw is uniform on its specified finite set, using fresh randomness conditional on the preceding choices. The implementation uses a fixed attempt limit. We couple it to the ideal experiment, rather than condition the whole experiment on every sampler succeeding: that latter conditioning could bias earlier adaptive choices.

Lemma 7.1 (Bounded samplers). *A uniform integer in $\{0, \dots, D-1\}$, with $1 \leq D < 2^w$, can be sampled using at most n^3 rejection attempts, or the trial can report failure. Conditional on any preceding history and on this call accepting, its result is uniform in the specified range. It couples to an ideal draw with failure probability at most 2^{-n^3} .*

For $4 \leq \ell \leq n$, the same attempt limit gives a uniform prime in $\mathcal{P}_\ell$, conditional on this call accepting, and coupling failure at most $e^{-n^2/4}$. A full trial, including permutations, signs, residues and the four separating families, couples to the ideal experiment with total failure $\exp(-\Omega(n^2))$. All constants are absolute. All permitted attempts are included in the deterministic work bounds.

Proof. For $D = 1$, return zero without randomness. Otherwise take $\lceil \log D \rceil$ low bits of a fresh random word and reject values at least D . Each value in the range has the same acceptance probability, and an attempt succeeds with probability at least $1/2$. The first accepted value is therefore uniform, including conditional on accepting within n^3 attempts. Comparing the same random attempts with an ideal repeat-until-success sampler gives failure probability at most 2^{-n^3} . Computing the bit length and each attempt takes polynomially many charged word operations. Uniform permutations use successive range draws among the remaining positions; a uniform fixed-size subset takes the required initial part of such a permutation. Sign masks use fresh unbiased bits.

For a prime, draw ℓ low bits, reject integers below two or outside $4\ell p \geq 2^\ell$, and test primality by deterministic trial division. Every prime in $\mathcal{P}_\ell$ has the same probability of being drawn. Lemma 2.3 gives acceptance probability at least $1/(4\ell)$, so the attempt limit fails with probability at most

$$(1 - 1/(4\ell))^{n^3} \leq e^{-n^3/(4\ell)} \leq e^{-n^2/4}.$$

The result is uniform conditional on this call accepting. In every application $\ell \leq .258n$, so trial division costs at most $\text{poly}(n)2^{.129n}$ operations per attempt, and its n^3 attempts preserve that form of bound.

The four families are the only exponentially numerous sampled objects. Each family has at most $10 \cdot 2^g + 1$ sets, each generated by at most g range calls. All other permutations, guesses and residues use polynomially many range calls, and there are constantly many prime calls. Thus each trial uses at most $\text{poly}(n)2^g$ range calls. The failure bounds hold conditional on every history, including adaptively chosen ranges. Summing them over the bounded number of calls gives

$$\text{poly}(n)2^g 2^{-n^3} + O(1)e^{-n^2/4} = \exp(-\Omega(n^2)).$$

This bounds the event that the bounded and ideal executions first diverge at an exhausted sampler. Retaining only current sampler state uses no uncounted random tape. The bounded cost, including all rejected attempts, is the cost charged by the trial analyses. $\square$

On the main branch, $\log u \leq n/10^9$, so the coupling loss is smaller than any fixed multiple of u^{-110} for large n . A watchdog interruption does not create an additional probabilistic assumption. On successful coupling, all sampled choices agree with the ideal choices, while the cost estimates already allow the worst permitted number of attempts. The favorable work events therefore complete before the watchdog; other executions may simply abort.

7.2 Exact word operations

The program scans the input and counts magnitude bits by shifts to obtain b . Actual signed weights, the transformed target and all relevant differences have magnitude less than

$$2^{b+\lceil \log(n+2) \rceil + 4}.$$

Even an invalid parent tuple uses an input index at most twice, so the same bound covers the quantities considered before disjointness is checked. Store a sign and a magnitude, with canonical zero. Signed addition, comparison and normalized remainders are implemented on these fields. When combining an actual weight with a residue, reduce the weight first. A fixed-width signed weight followed by the relevant mask bits supplies the trie key.

Every mask has at most n bits, and every stored item has a constant number of word fields. Binomial coefficients on at most n elements can be computed multiplicatively; immediately before division the intermediate integer is at most $n2^n$. Floor logarithms are bit lengths. Hence the background allocations, modulus parameters and rational floors are exact integer computations. The separator quantity 3^m fits in a word as well. Feasibility tests precede unsigned subtractions, or those quantities are represented with a sign. No entropy or real-arithmetic oracle is used by the program.

The largest product modulus satisfies $P \leq 2^{s+258n}$. Residue arithmetic therefore fits in a word, and trial division compares the candidate divisor with the corresponding integer quotient instead of forming a potentially unnecessary square. Masks for range sampling use fewer than w bits. The main-branch guard also gives

$$\log(u^{500}2^{\lceil n/2 \rceil}) \leq (1/2 + 500/10^9)n + 1 < w,$$

for large n . Thus fuel, the u^{120} round count, intermediate powers, timestamps and counters have bounded word width. The compact memory addresses fit by the space calculation below. All arithmetic invoked as a unit-cost operation stays within the stated word model.

Permutations, sign masks, moduli, residues, family masks and replay state remain in the counted workspace whenever reused. Once a random value is discarded, the program never reads it again.

7.3 Space on every random outcome

We specify the simultaneous storage terms, rather than infer space from the favorable resource event. Let $L_{\max} = 2^{185n}$, an upper bound on every raw representation leaf list on the main branch. There are only eight such lists; an item, heap cell or stream-row state occupies constantly many words. Linear auxiliary arrays suffice for sorting. Hence the stored leaves, heaps, row states and a constant number of replay snapshots occupy $O(L_{\max})$ words. The tie buffer uses $O(B)$ words. Initial compressed-block lists and mixer-counting half lists have sizes $2^s \leq 2^{12n}$; the compressed trial's stored sublists have size at most 2^{14n} . These fit the same bound, and the two trials run sequentially.

A dictionary keeps at most C distinct records and stops when a $(C+1)$ st key is encountered. Its fixed-width keys have $O(u)$ bits, so the binary trie, including transient nodes for that rejected key,

has $O(u(C + 1))$ nodes. Each node has constantly many word fields. At most a constant number of dictionaries and retained parent arrays coexist. In the right prefix search, an overflowing task's workspace is discarded before its children are processed. An $O(n)$ -depth stack stores prefixes and return information, not all previous task dictionaries. The completed left array is retained while the current right task or final check runs.

For the final check, the four family-index trees have total active depth $O(n)$; every depth keeps constantly many arrays of length at most C . These are actual $O(nC)$ retained record buffers, including linear sorting workspace, as in (26). Caller-retained parent arrays contribute another $O(C)$, already covered by this bound. The incidence tables have eight lists per block mask across the four blocks, each of length at most $10 \cdot 2^g + 1$. Their total capacity is at most

$$8 \cdot 2^g (10 \cdot 2^g + 1) = O(2^{2g}).$$

Each stored index uses at most $g + 4$ bits, and each family mask uses g bits, so both fit in one word. Directories and sampled families add only $O(2^g)$ words. The incidence contribution has the smaller exponential factor $2^{2g} \leq 2^{.12n}$.

Combination generation, permutations, sampler state, integer parameters, prefix and interval control, counters, and a conservative binomial workspace use $O(u^2)$ further words. In particular, none retains the hypothetical check list. Combining all terms, some absolute implementation constant K bounds simultaneous storage by

$$K[u(C + 1) + (n + 1)C + L_{\max} + B + 2^{2g} + u^2].$$

Now $C \leq 2^{.199n}$, $L_{\max} \leq 2^{.199n}$, $B \leq 2^{.199n}$, $2^{2g} \leq 2^{.199n}$, and $n \leq u$. Term by term this proves the required conservative bound

$$O(u^{15} 2^{.199n}) \tag{44}$$

on *every* random outcome, including an overflowing or aborted trial. No assumption about average bucket size or $\mathcal{E}$ was used. Its address bit length is at most $.199n + 15 \log u + O(1) < w$ on the main branch. Allocation and stack pointers are reset between tasks and trials; entries are initialized when reused. Even clearing the allocated storage linearly is below the trial time budget, so aborts accumulate neither stored data nor uncharged cleanup work.

7.4 A finite program, amplification and the theorem

There are finitely many sufficiently-large conditions in the argument, with absolute constants and fixed polynomial degrees. The main-branch guard makes each fixed exponential margin dominate the required polynomial factors uniformly in that branch. Choose one absolute integer n_0 meeting all these conditions and fix it in the program, independently of c .

Small-word inputs need not hold n_0 as a word. Place an exact subset-enumeration routine near the beginning of the finite program. Unroll the binary digits of n_0 , from least to most significant, using only literals zero and one. Maintain a shifted copy of n and a three-valued comparison state. At each digit, compare its low bit with the fixed threshold digit, replace the comparison state when they differ, and shift the copy right. A newly seen difference has higher significance. If the copy becomes zero before the threshold's leading digit has been processed, branch to the nearby exhaustive routine. After that leading digit, a nonzero remaining copy means $n > n_0$; otherwise the comparison state decides.

A short input executes only $O(1 + \log n)$ adjacent digit stages before exiting. Its instruction addresses and state therefore remain small; it never loads the full threshold or a distant main-program address. Once $n \geq n_0$, enlarge this fixed threshold if necessary so that all other fixed

literals and main-program addresses fit. The second guard, (2), is then an integer comparison. Both exceptional cases use exact enumeration with an n -bit subset counter and $O(n)$ words. This is one finite program with no advice and no input for the exponent c .

On a main-branch YES input, Lemma 3.1 gives constant success probability when $\mu \geq .00765n$, and Lemma 6.2 gives probability at least u^{-110} when $\mu < .00765n$. Both trials run in each round with fresh words and reset state. Thus each round succeeds with probability at least u^{-110} , also conditional on every previous round failing. After u^{120} rounds the probability of missing a solution is at most

$$(1 - u^{-110})^{u^{120}} \leq e^{-u^{10}} < 1/3.$$

Every YES is an exact valid match by Lemma 4.1; the sign-toggle identity certifies an original solution. The exceptional branch is exact, so correctness holds on every individual input.

Instrument each trial with a counter for underlying word operations. Its constant overhead, stopping and reset give per-trial time $O(u^{500}2^{\lceil n/2 \rceil})$ on every outcome. Input scanning and the initial guards take polynomial time in u . All trial preprocessing, sorting, table construction and sampling are charged. The total main-branch time is consequently

$$O(u^{620}2^{\lceil n/2 \rceil} + \text{poly}(u)).$$

For every fixed positive integer c , all sufficiently large inputs with $b \leq n^c$ avoid the exceptional branch and have $u = \text{poly}_c(n)$. This time bound and (44) hold simultaneously for every random outcome. Together with the correctness argument they prove Theorem 1.1. Finally, for each fixed $\rho > .199$, the factor $u^{15} = \text{poly}_c(n)$ is absorbed by $2^{(\rho-.199)n}$, proving Corollary 1.2; the polynomial factor remains at the endpoint .199.

References

- [1] Per Austrin, Petteri Kaski, Mikko Koivisto, and Jussi Määttä, *Space–Time Tradeoffs for Subset Sum: An Improved Worst Case Algorithm*, Automata, Languages, and Programming (ICALP 2013), Part I, Lecture Notes in Computer Science **7965**, Springer, 2013, 45–56. doi:10.1007/978-3-642-39206-1_5.
- [2] Per Austrin, Petteri Kaski, Mikko Koivisto, and Jesper Nederlof, *Dense Subset Sum May Be the Hardest*, 33rd Symposium on Theoretical Aspects of Computer Science (STACS 2016), Leibniz International Proceedings in Informatics **47** (2016), 13:1–13:14. doi:10.4230/LIPIcs.STACS.2016.13.
- [3] Tatiana Belova, Nikolai Chukhin, Alexander S. Kulikov, and Ivan Mihajlin, *Improved Space Bounds for Subset Sum*, 32nd Annual European Symposium on Algorithms (ESA 2024), Leibniz International Proceedings in Informatics **308** (2024), 21:1–21:17. doi:10.4230/LIPIcs.ESA.2024.21. Full version, arXiv:2402.13170v3, August 1, 2024. arXiv:2402.13170v3.
- [4] Xi Chen, Yaonan Jin, Tim Randolph, and Rocco A. Servedio, *Subset Sum in Time $2^{n/2}/\text{poly}(n)$* , Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2023), Leibniz International Proceedings in Informatics **275** (2023), 39:1–39:18. doi:10.4230/LIPIcs.APPROX/RANDOM.2023.39. Full version, arXiv:2301.07134v2, January 29, 2023. arXiv:2301.07134v2.

- [5] Itai Dinur, Orr Dunkelman, Nathan Keller, and Adi Shamir, *Efficient Dissection of Composite Problems, with Applications to Cryptanalysis, Knapsacks, and Combinatorial Search Problems*, Advances in Cryptology—CRYPTO 2012, Lecture Notes in Computer Science **7417**, Springer, 2012, 719–740. doi:10.1007/978-3-642-32009-5_42.
- [6] Fedor V. Fomin, Daniel Lokshtanov, Fahad Panolan, and Saket Saurabh, *Efficient Computation of Representative Families with Applications in Parameterized and Exact Algorithms*, Journal of the ACM **63** (2016), no. 4, Article 29, 29:1–29:60. doi:10.1145/2886094.
- [7] Nick Howgrave-Graham and Antoine Joux, *New Generic Algorithms for Hard Knapsacks*, Advances in Cryptology—EUROCRYPT 2010, Lecture Notes in Computer Science **6110**, Springer, 2010, 235–256. doi:10.1007/978-3-642-13190-5_12.
- [8] Ellis Horowitz and Sartaj Sahni, *Computing Partitions with Applications to the Knapsack Problem*, Journal of the ACM **21** (1974), no. 2, 277–292. doi:10.1145/321812.321823.
- [9] Jesper Nederlof and Karol Węgrzycki, *Improving Schroeppeel and Shamir’s Algorithm for Subset Sum via Orthogonal Vectors*, Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021), 1670–1683. doi:10.1145/3406325.3451024. Full version, arXiv:2010.08576v2, April 12, 2021. arXiv:2010.08576v2.
- [10] Richard Schroeppeel and Adi Shamir, *A $T = O(2^{n/2})$, $S = O(2^{n/4})$ Algorithm for Certain NP-Complete Problems*, SIAM Journal on Computing **10** (1981), no. 3, 456–464. doi:10.1137/0210033.